

9

Tuples and Dictionary

9.1 INTRODUCTION

While writing programs or working with the programming languages, we often come across a term called Data Structure. Data structures are the structures that hold and organize data or information. In the previous chapter, we discussed about lists (a sequence) which are nothing but data structures that store related data items.

In this chapter, we will focus on another important Python sequence, viz. Tuples and an ordered sequence/collection called Dictionary.

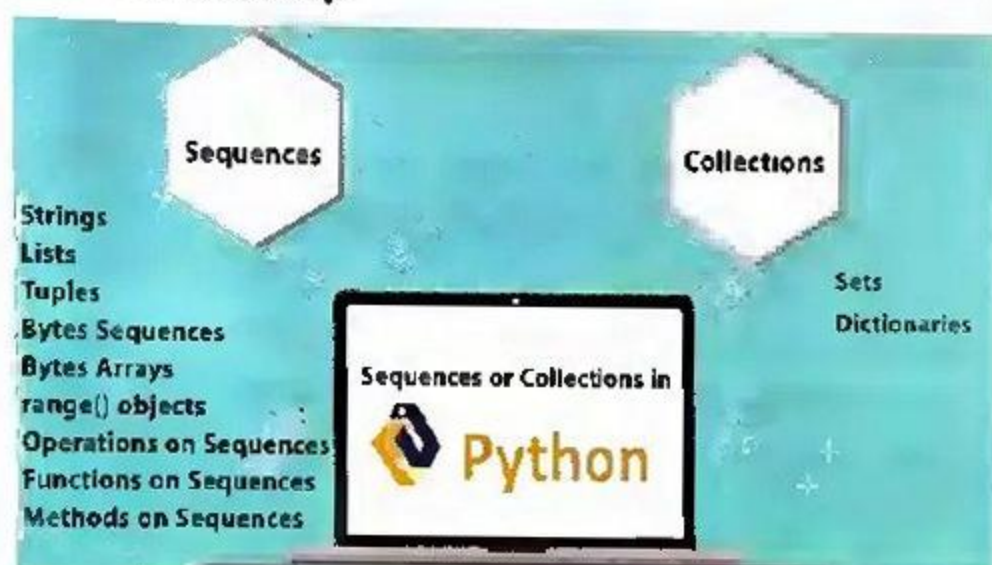


Fig. 9.1: Classification of Sequences/Collections in Python

Sequence or say a collection, with which we are already familiar, contains several items or operations performed one after the other. A traffic light sequence is the most common example of a sequence which we encounter in our daily life.

Python has several types of sequences such as lists, tuples, strings, etc. All these share the following common features:

- Each sequence contains a finite number of elements.
- The elements are arranged in some order in the sequence.
- It is possible to iterate over the elements in the sequence, according to their order.
- It is also possible to check if an element is in a sequence or not.

Let us discuss tuples and dictionary in detail.

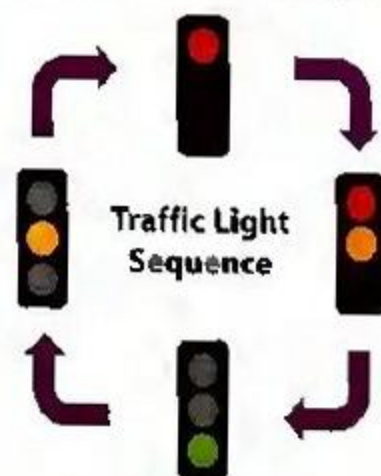


Fig. 9.2: A Common Sequence in Daily Life

9.2 TUPLES

Tuple is a data structure in Python. It is another sequence data type that is similar to the list. A tuple consists of multiple values in a single variable separated by commas. Unlike lists, however, tuples are enclosed within parentheses() as shown in Fig. 9.3

("Mango", "Apples", "Grapes")



Fig. 9.3: A Tuple in Python

The values that make up a tuple are called its elements. Tuples allows duplicate values. Elements in a tuple need not be of the same type, they can be heterogeneous, i.e., of different data types. Thus, we may say that tuples are the comma-separated sequence of values declared using parentheses (). Its syntax is:

<tuple_name/variable name> = (<value1>, <value2>, ..., <value n>)

For example,

```
Fruits = ('Mango', 'Apple', 'Grapes')
```

In the above example, we have a tuple named 'Fruits', with three elements as Mango, Apple and Grapes. The elements in a tuple are addressed using their index value. The index value of tuple starts with 0.

The main differences between lists and tuples are: Lists are enclosed within square brackets [], and their elements and size can be changed, while tuples are enclosed within parentheses () and cannot be updated. Tuples can be thought of as read-only lists.

Tuples are immutable, i.e., we cannot change the elements of a tuple in place. This means we cannot perform insert, update and delete operations on them. That is why tuples are faster and more efficient than lists.

CTM: Tuples are immutable, but member objects may be mutable.

Example 1:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bi
t (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> digits= (0,1,2,3,4,5,6,7,)
>>> tup= ('hello',1,5,6.5,'Python') #Heterogeneous tuple
>>> tup = () # an empty tuple
>>> tup = (10,20,30)
>>> tup = 10,20,30 #parentheses are not must
>>> tup = ((0,1,2,3),(4,5,6)) #nested tuple- tuples inside a tuple
>>> tup = ((0,1,2,3),['hello','world']) # tuple and list inside a tuple -
a nested tuple
>>> tup= ([1,2,3],[4,5,6]) #lists within a tuple
>>> tup= ('a','b','c',[10,20,30]) #a tuple containing strings and list
```

Syntactically, a tuple is a comma-separated list of values:

```
>>> tup = 'a', 'b', 'c', 'd', 'e'
>>> tup = (70,)
# tuple with single item
```

➤ If a tuple comprises a single element, the element should be followed by a comma to distinguish a tuple from a parenthesized expression.

➤ A tuple having a single element is also known as **singleton tuple**

➤ **Tuples being immutable**, an attempt to modify an element of a tuple yields an error, as shown in Fig. 9.4.

Learning Tip: If the contents of a list shouldn't change, use a tuple to prevent items from accidentally being added, changed or deleted.

```
>>> tup=(10,20,30,40)
>>> tup[2]=100
Traceback (most recent call last):
  File "<pyshell#28>", line 1, in <module>
    tup[2]=100
TypeError: 'tuple' object does not support item assignment
>>> tup1=(10,20,30,40,[4,5,6])
>>> tup1[4][1]=500
>>> tup1
(10, 20, 30, 40, [4, 500, 6])
>>>
```

Fig. 9.4: Modifying a Tuple Element Generates an Error

CTM: List is mutable but tuple is immutable. So, iterating through a tuple is faster as compared to a list. If we have data that is not required to be changed, then storing this data in a tuple will ensure that it is not changed accidentally.

However, if an element of a tuple is of mutable type, *e.g.*, a list, then we can modify that element.

9.3 TUPLE CREATION

A tuple is enclosed in parentheses/round brackets () for creation and each item is separated by a comma. Round brackets indicate the start and end of the tuple. The syntax to create a tuple is:

Syntax: Tup=()

Tup=(value1, value2..... value n)

For example,

```
tup1 = ('compsc', 'infopractices', 2017, 2018)
```

```
tup2 = (5, 11, 22, 44)
```

The above construct is known as **tuple display construct**.

tuple with mixed data types

```
>>> my_tuple = (1, "Hello", 3.4)
```

```
>>> print(my_tuple)
```

```
(1, 'Hello', 3.4)
```

tuple having strings

```
>>> thistuple = ('apple', 'banana', 'cherry')
```

```
>>> print(thistuple)
```

```
('apple', 'banana', 'cherry')
```

CTM: Indexing of tuple is just like indexing of list.

1. Creating tuple using built-in function tuple()

An empty tuple can be created using tuple() function.

Syntax:

2. Creating tuple with a single element

To create a tuple with a single element, you have to include the final comma. The final comma after single value is mandatory otherwise it is not considered as tuple, rather treated as a character or a string or an integer.

```
Python 3.7.4 Shell
>>> t1 = tuple()
>>> type(t1)
<class 'tuple'>
>>> t2 = ('a',)
>>> type(t2)
<class 'tuple'>
>>> t3 = (10)
>>> type(t3)
<class 'int'>
```

Example 2: to create empty tuple using function tuple().

```
T = tuple()
print(T)
```

Example 3: to create tuple containing string values using function tuple().

```
>>> tuple("string",)
('s', 't', 'r', 'i', 'n', 'g')
>>> mytuple = tuple(("apple", "banana", "cherry"))
>>> print(mytuple)
('apple', 'banana', 'cherry')
>>> type(mytuple)
<class 'tuple'>
```

The function tuple() can be used to convert a sequence into a tuple.

3. Creating tuple by accepting user input using while loop

We can create a tuple by fetching its elements from the user. It can be done either by while/for loop.

practical Implementation-1

Write a Python code to create a tuple by accepting elements from the user using *while* loop.

```
prgm_1.py - C:/Chapter 8/prgm_1.py (3.6.5)
File Edit Format Run Options Window Help
t=tuple()
n=int(input("Enter any number: "))
i=1
while (i<=n):
    a=input("Enter Number: ")
    t=t+(a,)
    i=i+1
print("Output is")
print(t)
```

Ln 10 Col:6

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Chapter 8/prgm_1.py =====
Enter any number: 3
Enter Number: 100
Enter Number: 200
Enter Number: 300
Output is
('100', '200', '300')
>>>
```

Ln 11 Col:4

Practical Implementation-2

Modify the above program using *for* loop.

```
prgm_2.py - C:/Chapter 8/prgm_2.py (3.6.5)
File Edit Format Run Options Window Help
t=tuple()
n=int(input("Enter any number: "))
for i in range(n):
    a=input("Enter Number: ")
    t=t+(a,)
print("Output is")
print(t)
```

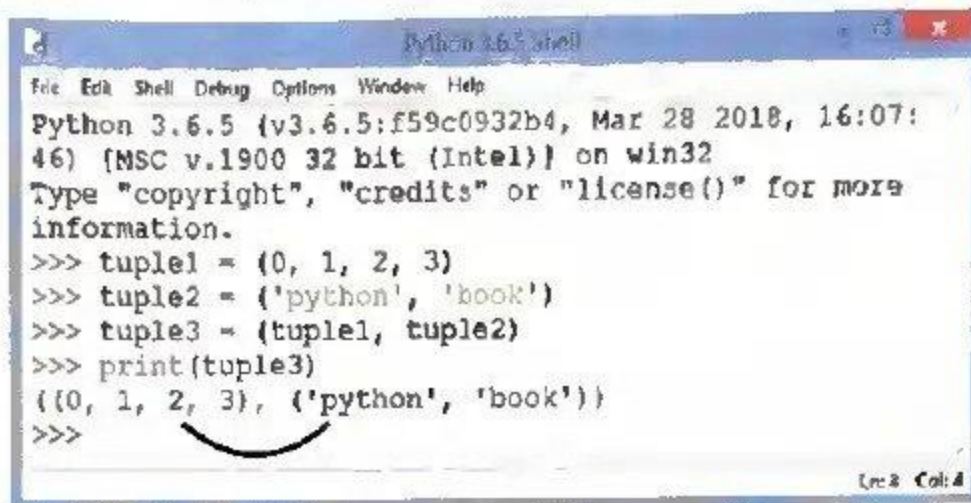
Ln 8 Col:0

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Chapter 8/prgm_2.py =====
Enter any number: 4
Enter Number: 50
Enter Number: 20
Enter Number: 80
Enter Number: 100
Output is
('50', '20', '80', '100')
>>>
```

Ln 12 Col:4

9.4 NESTING OF TUPLES

Tuples can be placed inside other tuples. When you add one or more tuples inside another tuple, the items in the nested tuples are combined together to form a new tuple.

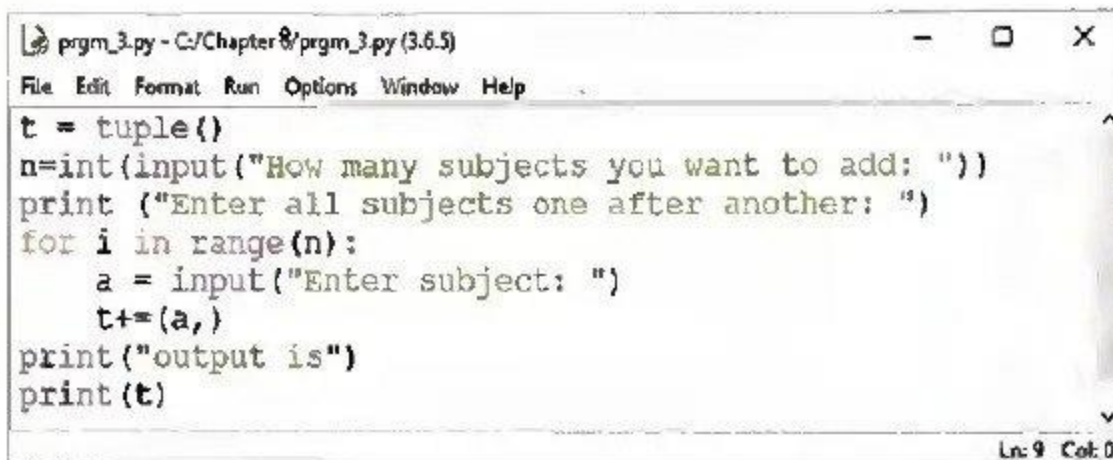


```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> tuple1 = (0, 1, 2, 3)
>>> tuple2 = ('python', 'book')
>>> tuple3 = (tuple1, tuple2)
>>> print(tuple3)
((0, 1, 2, 3), ('python', 'book'))
>>>
```

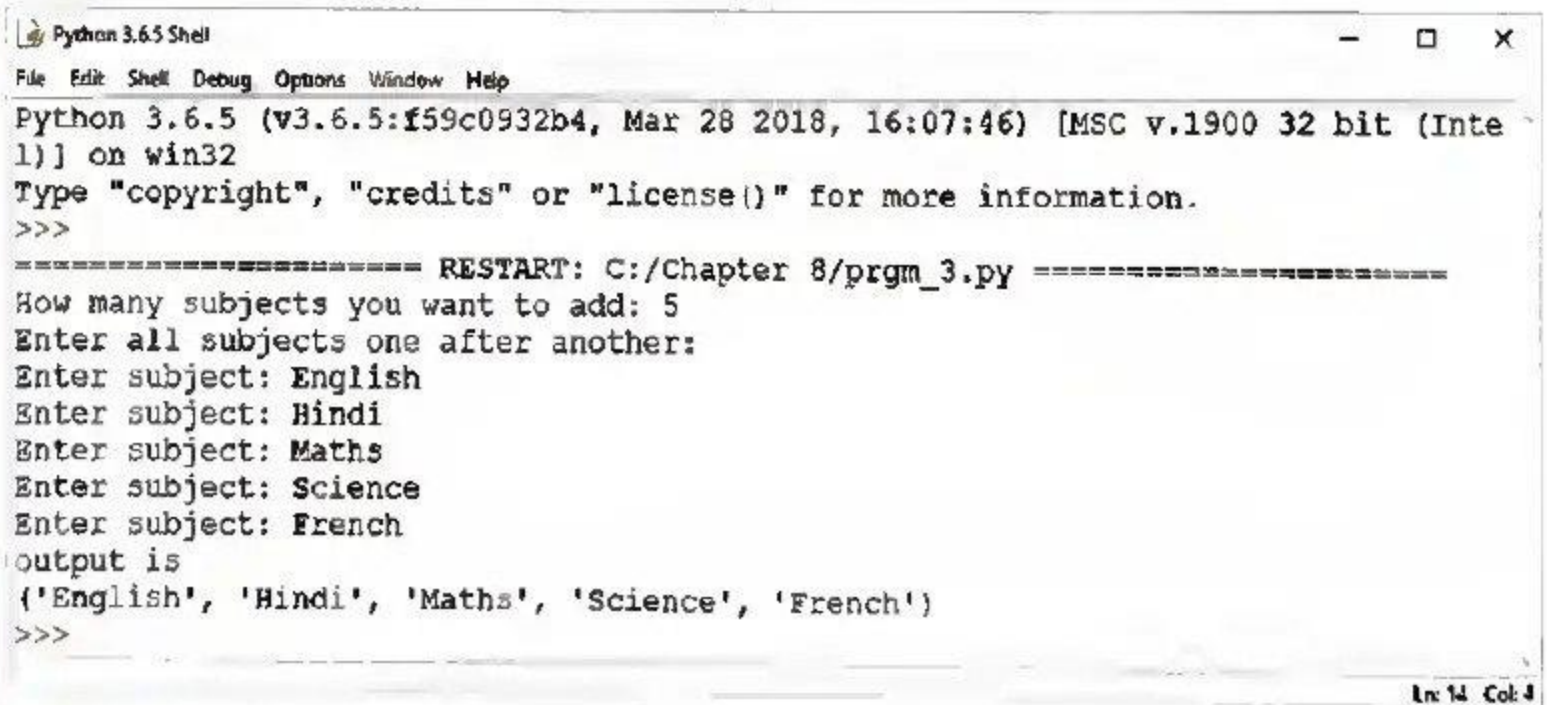
As shown in the above example, tuple1 and tuple2 are combined to form a new tuple as tuple3.

Practical Implementation-3

Write a Python code to store 'n' number of subjects in a tuple.



```
prgm_3.py - C:/Chapter8/prgm_3.py (3.6.5)
File Edit Format Run Options Window Help
t = tuple()
n=int(input("How many subjects you want to add: "))
print ("Enter all subjects one after another: ")
for i in range(n):
    a = input("Enter subject: ")
    t+=(a,)
print("output is")
print(t)
```



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Chapter 8/prgm_3.py =====
How many subjects you want to add: 5
Enter all subjects one after another:
Enter subject: English
Enter subject: Hindi
Enter subject: Maths
Enter subject: Science
Enter subject: French
output is
('English', 'Hindi', 'Maths', 'Science', 'French')
>>>
```

Practical Implementation-4.

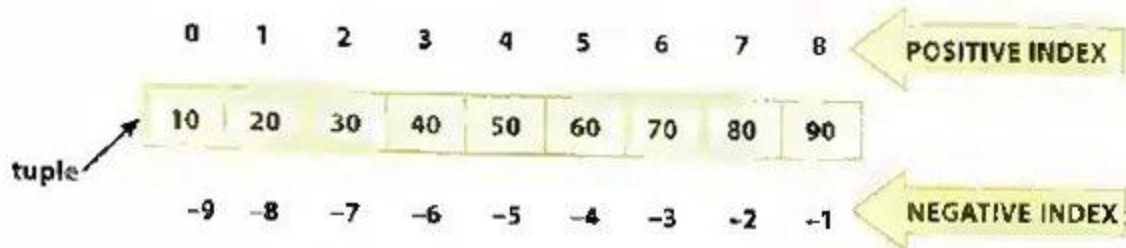
Write a program to create a nested tuple to store roll number, name and marks of students.

```
#Program to store records of students in tuple and print them
st=((200,"Harmeet",88),(201,"Deepika",98),(202,"Radhika",78),(204,"Shaurya",90))
print("S_No","Roll_No","Name","Marks")
for i in range(0,len(st)):
    print((i+1),'\t',st[i][0],'\t',st[i][1],'\t',st[i][2])
```

```
>>>
RESTART: C:/Users/preeti/AppData/Local
tup.py
S_No  Roll_No  Name      Marks
1      200     Harmeet    88
2      201     Deepika    98
3      202     Radhika    78
4      204     Shaurya    90
```

9.5 ACCESSING AND TRAVERSING A TUPLE

A tuple is a sequence of values which can be of any type and they are indexed by integer. Like lists, there could be positive indexing like 0, 1, 2, 3 ... or negative indexing like -1, -2, -3 ..., etc.



The individual elements of a **tuple** can be accessed through their indexes given in square brackets as shown in Fig. 9.5.

Elements of a tuple can be accessed using an **indexing** method.

Tuple indices start at 0.

The tuple index must be a positive or negative integer value or an expression which evaluates to an integer value. Positive value of index means counting forward from beginning of the tuple and negative value means counting backward from end of the tuple.

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:
46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
>>> t1=(10,20,30,40,50,60,70,80,90)
>>> t1[0]
10
>>> t1[5]
60
>>> t1[-5]
50
Ln: 8 Col: 20
```

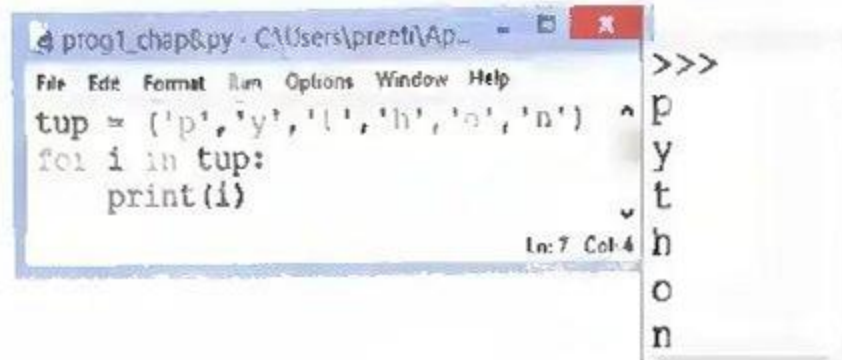
Fig. 9.5: Accessing a Tuple Element

Traversing a Tuple

Traversing a tuple means accessing each element of a tuple one after the other at the same time. This can be done by using either *for* or *while* loop.

1. Using in operator using for loop

'in' operator used with 'for' loop iterates each element of a tuple in sequence.



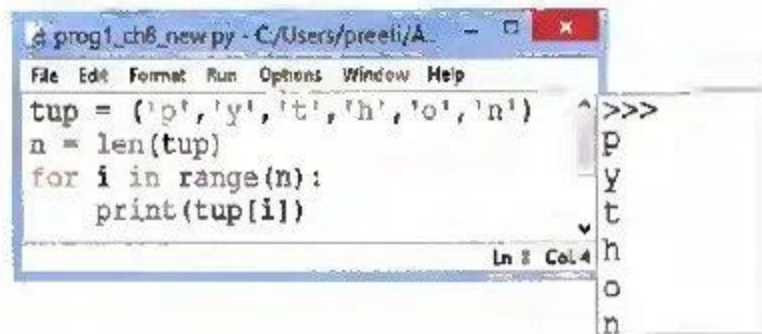
```

prog1_chap8.py - C:\Users\preeti\Ap...
File Edit Format Run Options Window Help
tup = ('p','y','t','h','o','n')
for i in tup:
    print(i)
Ln: 7 Col: 4
>>>
p
y
t
h
o
n
  
```

In the above program, the loop variable 'i' shall be assigned the tuple elements one at a time. The loop variable 'i' will be assigned 'p' in the first iteration and, hence, 'p' gets printed. Similarly, in the second iteration, 'y' gets accessed and printed. This process continues till all the elements of the tuple 'tup' are accessed and displayed.

2. Using range() function

Like lists, tuples also can be created using range() function.



```

prog1_ch8_new.py - C:\Users\preeti\A...
File Edit Format Run Options Window Help
tup = ('p','y','t','h','o','n')
n = len(tup)
for i in range(n):
    print(tup[i])
Ln: 8 Col: 4
>>>
p
y
t
h
o
n
  
```

When the above code gets executed, loop variable 'i' will be assigned values from 0 to n-1, i.e., 5. In the first iteration, it will print tup[0], i.e., 'p' then tup[1], i.e., 'y' and so on up to tup[5], i.e., 'n'.

9.6 COMMON TUPLE OPERATIONS

Everything else works with tuple as it did with lists, like indexing, getting the length, etc. All of the common sequence operations are available with tuples also.

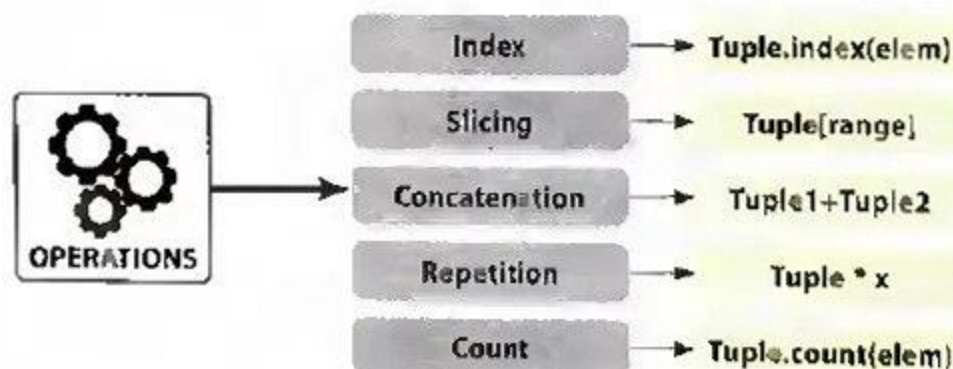


Fig. 9.6: Common Tuple Operations

Let us discuss each of these operations:

9.6.1 Tuple Slicing

Slicing is used to retrieve a subset of values. A slice of a tuple is basically its sub-tuple (Fig. 9.7).

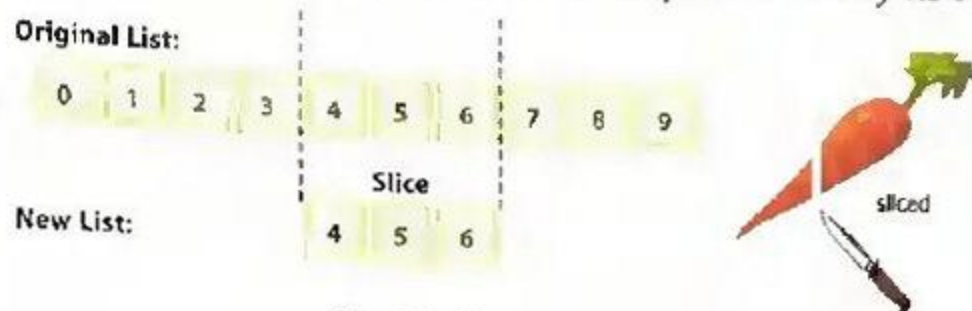


Fig. 9.7: Slicing in Tuple

Syntax:

```
tuple_name[start:stop: step]
```

where,

start is the starting point

stop is the stopping point, which is excluded

step is the step size—also known as stride

If you omit first index, slice starts from 0th index value and omission of stop will take it to the end. Default value of step is 1.

However, it includes the first element but excludes the last element.

The first value in a tuple is at indexed 0. Just like lists, you can use the index values in combination with square brackets [] to access items in a tuple:

	0	1	2	3	4	5	6	7	8
	10	20	30	40	50	60	70	80	90
tuple1	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
===== RESTART: Shell =====
>>> tuple1 = (10,20,30,40,50,60,70,80,90)
>>> tuple1[5:]
(60, 70, 80, 90)
>>>
```

```
>>> T1 = ('MY BOOK', [1, 2, 3,] 'Z', (3, 4, 6), 'DESK', 50)
```

```
>>> T1 [2:3]
```

```
('Z',)
```

```
>>> T1 [1:2]
```

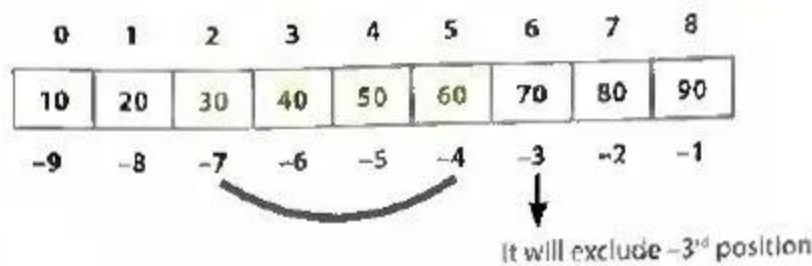
```
([1, 2, 3],)
```

```
>>> T1[3][1]
```

```
4
```

A tuple element can be accessed and traversed inversely also, i.e., in reverse direction.

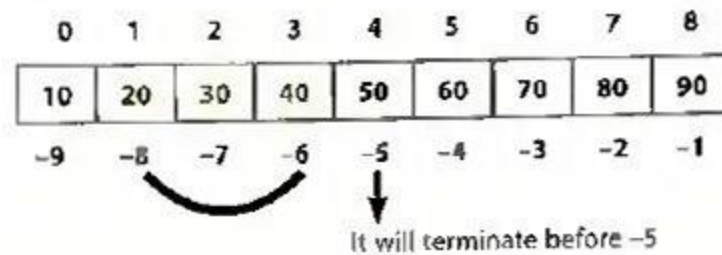
```
>>> tuple1 = (10, 20, 30, 40, 50, 60, 70, 80, 90)
>>> tuple1[-7:-3]
```



Output:

```
(30, 40, 50, 60)
```

```
>>> tuple1[-8:-5]
```



Output:

```
(20, 30, 40)
```

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:0
7:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for mor
e information.
>>> tuple1[::-1]
(90, 80, 70, 60, 50, 40, 30, 20, 10)
```

In the above example, the slicing range for tuple 'tuple1' includes -1 as the step value, but the start and end values are omitted. Hence, it will extract all the elements from the tuple, starting from index 0 to the last index, but in reverse order. Thus, the above slice range will reverse the tuple and the output is as obtained.

- **Slice Steps:** You can also extract non-consecutive elements from a tuple by changing the value of argument 'step'.

For example,

```
>>> tuple1 = (10, 20, 30, 40, 50, 60, 70, 80, 90)
>>> tuple1
(10, 20, 30, 40, 50, 60, 70, 80, 90)
>>> tuple1[0:10:2] #Accessing every 2nd element, skipping
1 element in between
(10, 30, 50, 70, 90)
>>> tuple1[1:10:3] #step value as 3 shall take every 3rd
element and skip 2 elements in between
(20, 50, 80)
>>> tuple1[::-3] #will extract every 3rd element from the
tuple. No start and stop value. only step value
(10, 40, 70)
>>> tuple1[::2] #only step value as 2
(10, 30, 50, 70, 90)
>>> tuple1[4::2] #Extracts tuple from 4th element to the
end of tuple with a step size of 2
(50, 70, 90)
```

9.6.2 Tuple Addition/Concatenation

python allows you to join/concatenate two tuples. New elements can be added to a tuple using '+' operator.

```
>>> x = (1, 2, 3, 4)
>>> y = (5, 6, 7, 8)

# Combining two tuples to form a new tuple
>>> z = x + y
>>> print (z)
(1, 2, 3, 4, 5, 6, 7, 8)
```

You can use '+' operator with tuple slice also.

```
>>> tup1=(1, 2, 3, 4)
>>> tup1[1:3]+(5,6)
(2, 3, 5, 6)
>>> t1=(1,2,'a',4)
>>> t1[-3:-1]+t1[0]    # will give error as you cannot add int to tuple.
Traceback (most recent call last):
  File "<pysHELL#49>", line 1, in <module>
    t1[-3:-1]+t1[0]
```

TypeError: can only concatenate tuple (not "int") to tuple

'+' operator performs concatenation with tuples only.

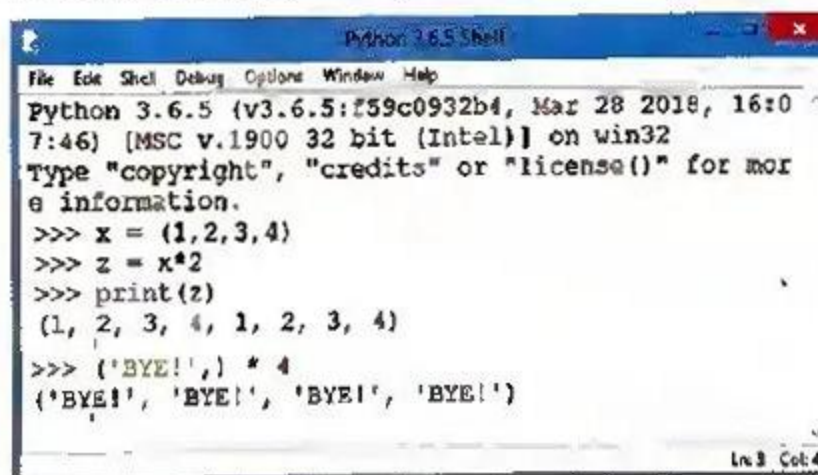
```
>>> t1[-3:-1]+(t1[0],)
(2, 'a', 1)
```

CTM: We can only add tuple to a tuple.

NOTE: A single value included in () is treated as a single value and not as a tuple. So, if you write (10) and ('a'), they are treated as integer and string respectively. But if they are given as (10,) and ('a,') these elements are treated as tuples with single element.

9.6.3 Tuple Multiplication/Repetition

The multiplication operator (*) is used for repetition of the tuple.



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:0
7:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for mor
e information.
>>> x = (1,2,3,4)
>>> z = x*2
>>> print(z)
(1, 2, 3, 4, 1, 2, 3, 4)
>>> ('BYE!') * 4
('BYE!', 'BYE!', 'BYE!', 'BYE!')
```

You can use '*' operator with tuple slice also.

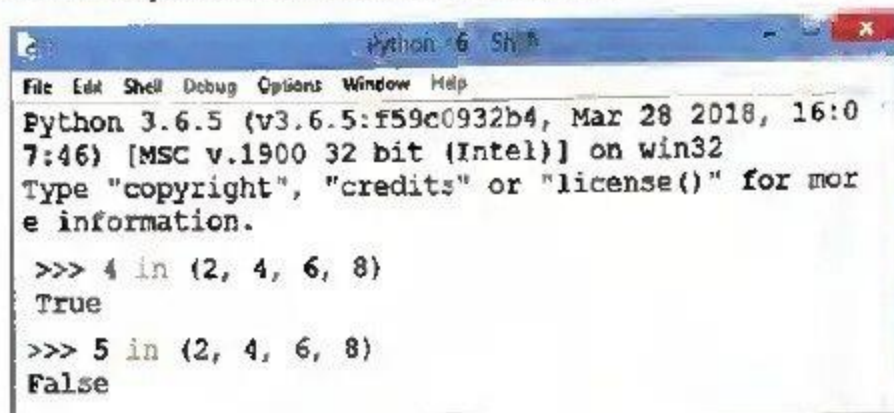
```
>>> tup1=(1,2,3,4)
>>> tup1[1:3]*3
(2, 3, 2, 3, 2, 3)
```

As is evident from the output obtained, the * operator has been used to replicate the tuple slice and not the full tuple.

CTM: '*' operator requires a tuple and an integer as operands.

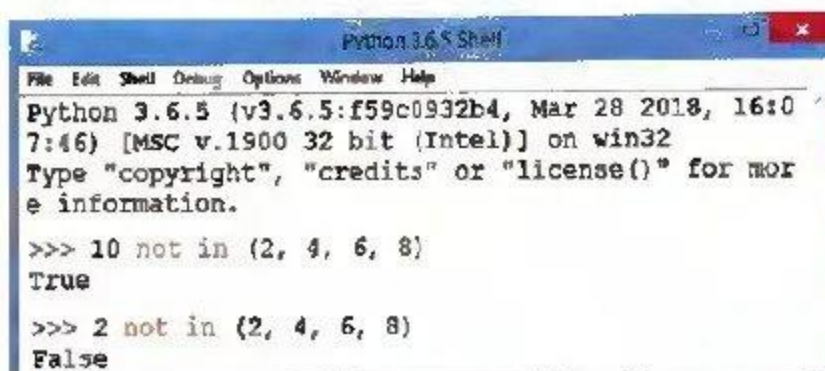
9.6.4 Membership Operator 'in' and 'not in'

The 'in' operator checks whether a given element is contained in a tuple. It returns true if element is present in the tuples, otherwise returns false.



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 4 in (2, 4, 6, 8)
True
>>> 5 in (2, 4, 6, 8)
False
```

'not in' operator returns true if element does not appear in the tuples, otherwise returns false.



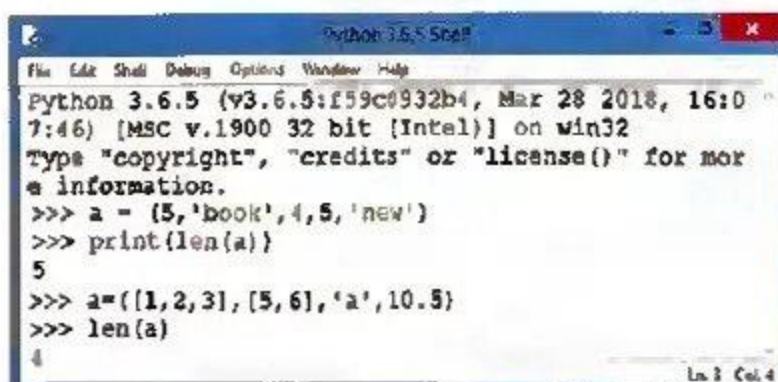
```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 10 not in (2, 4, 6, 8)
True
>>> 2 not in (2, 4, 6, 8)
False
```

Tuple Functions

Being immutable in nature unlike lists, tuples do not support methods such as append(), extend(), remove(), insert(), and pop() since all these operations require modifications to be made on a tuple which is not permitted.

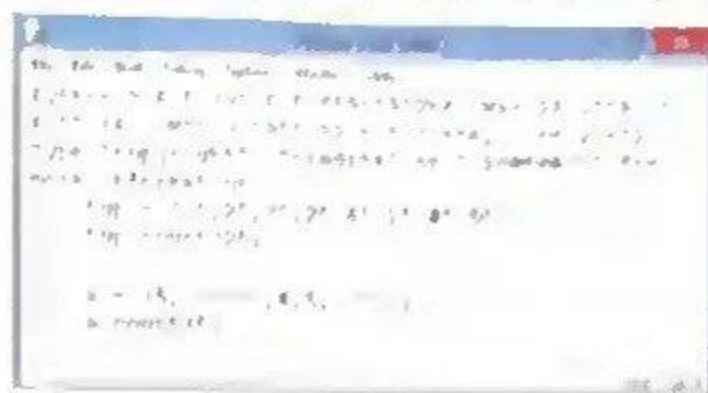
However, tuples work well with several built-in methods which are as follows:

1. **len():** This function returns the length of a tuple, i.e., count the total number of elements in a tuple.

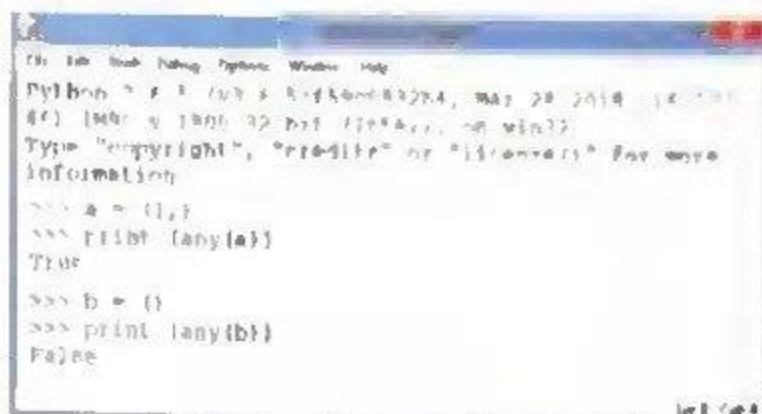


```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a = (5, 'book', 4, 5, 'new')
>>> print(len(a))
5
>>> a = ([1, 2, 3], [5, 6], 'a', 10.5)
>>> len(a)
4
```

2 **count():** This function is used to count the number of occurrences of an item in the tuple.



3. **any()**: This function returns True if a tuple is having at least one item. If the tuple is empty it will return False.



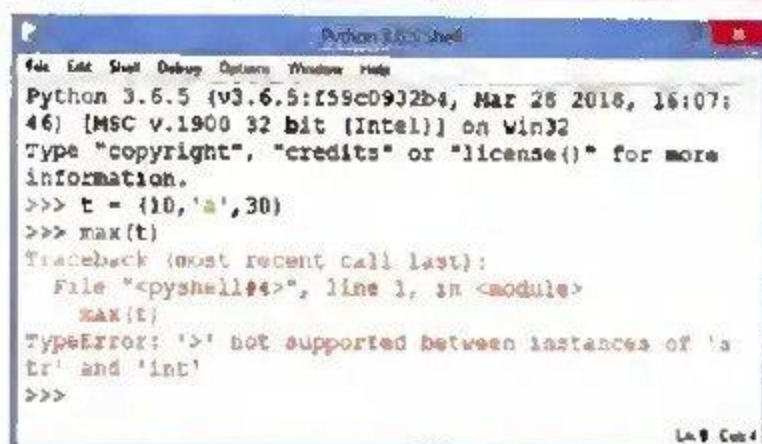
Since tuple b is an empty tuple, therefore, any() function shall return False as the output.

4. min() and max()

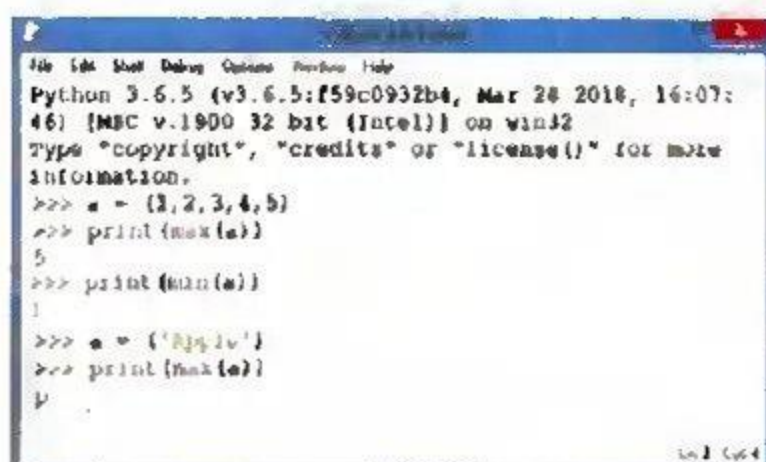
min() function shall return the element with minimum value from the tuple. On the other hand, **max()** function shall return the element with maximum value from the tuple

POINT TO REMEMBER

To use the min() and max() function, all values in the tuple must be of same type.

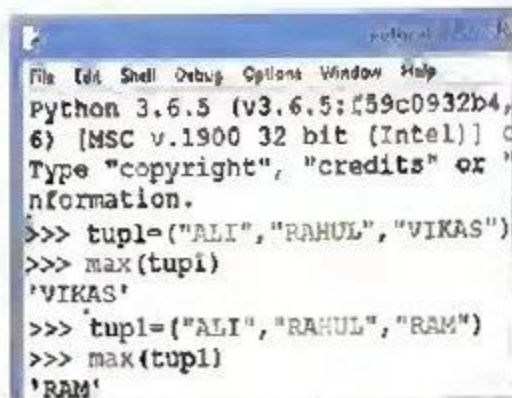


Since the tuple 't' holds values of different types, max() function generates an error as these functions work only with elements of same data type.



Both the functions work well with tuples containing string values as well. In the above example, the string 'Apple' gets converted into individual characters and are then compared to return the highest value.

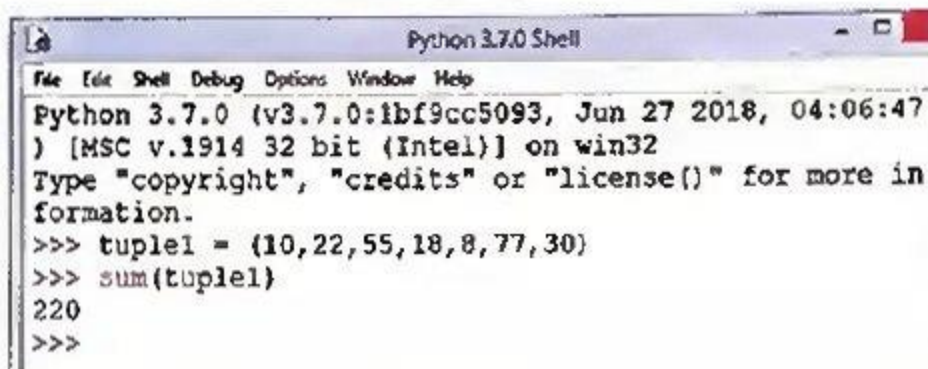
Note: `max()` and `min()` check for the characters in a string on the basis of their ASCII values. If there are two strings in a tuple, then the first different character in the strings is checked.



```
Python 3.6.5 (v3.6.5:f59c0932b4,
6) [MSC v.1900 32 bit (Intel)] c
Type "copyright", "credits" or "
nformation.
>>> tup1=("ALI","RAHUL","VIKAS")
>>> max(tup1)
'VIKAS'
>>> tup1=("ALI","RAHUL","RAM")
>>> max(tup1)
'RAM'
```

Explanation: Here, first two characters of 'RAHUL' and 'RAM' are same, so it will compare ASCII value of third character and will return 'RAM' as output.

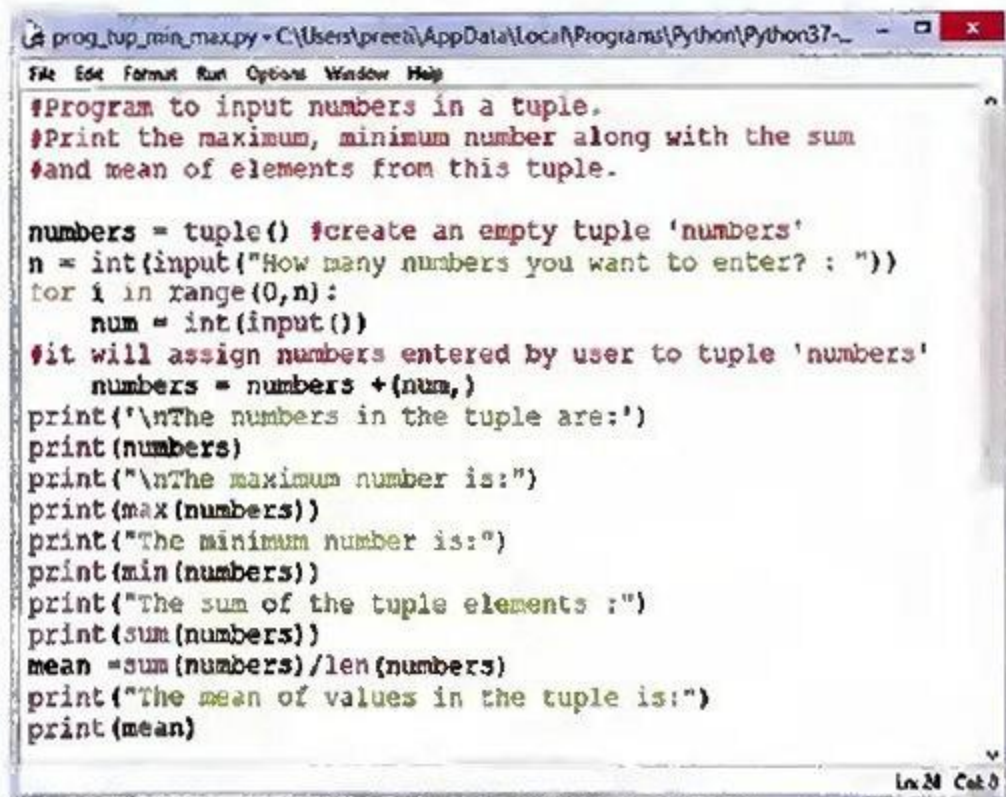
5. **sum():** This function returns sum of the elements of the tuple.



```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47
) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more in
formation.
>>> tuple1 = (10,22,55,18,8,77,30)
>>> sum(tuple1)
220
>>>
```

Practical Implementation-5

Write a program to input *n* numbers from the user, store these numbers in a tuple and print the maximum, minimum number along with the sum and mean of all the elements from this tuple.



```
prog_tup_min_max.py - C:\Users\preen\AppData\Local\Programs\Python\Python37-
File Edit Format Run Options Window Help
#Program to input numbers in a tuple.
#Print the maximum, minimum number along with the sum
#and mean of elements from this tuple.

numbers = tuple() #create an empty tuple 'numbers'
n = int(input("How many numbers you want to enter? : "))
for i in range(0,n):
    num = int(input())
    #it will assign numbers entered by user to tuple 'numbers'
    numbers = numbers +(num,)
print("\nThe numbers in the tuple are:")
print(numbers)
print("\nThe maximum number is:")
print(max(numbers))
print("The minimum number is:")
print(min(numbers))
print("The sum of the tuple elements :")
print(sum(numbers))
mean =sum(numbers)/len(numbers)
print("The mean of values in the tuple is:")
print(mean)
```

```

>>>
RESTART: C:\Users\preeti\AppData\Local\Programs\
max.py
How many numbers you want to enter? : 6
10
20
30
40
50
60

The numbers in the tuple are:
(10, 20, 30, 40, 50, 60)

The maximum number is:
60
The minimum number is:
10
The sum of the tuple elements :
210
The mean of values in the tuple is:
35.0
>>>

```

6. sorted()

sorted() is used to sort the elements of a tuple. It returns a list after sorting.

For example,

```

>>> a = (6, 7, 4, 2, 1, 5, 3)
>>> sorted(a)
[1, 2, 3, 4, 5, 6, 7]

```

It is worth noting that the return type is a list and not a tuple. The sequence in the original tuple 'a' is not changed and the data type of 'a' remains tuple.

7. index()

index() function finds the first index of a given item and returns the index.

Syntax:

```
tuple.index(value, start, end)
```

Here, start and end are two optional parameters for the start and end of the tuple's index.

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:0
32 bit (Intel)) on win32
Type "copyright", "credits" or "license()" for mor
>>> vote = ("yes", "no", "yes", "yes", "no")
>>> vote.index("no")
1
>>> vote.index("yes")
0
>>> vote.index("yes", 1)
2
>>> vote.index("no", 2)
4
>>> vote.index("yes", 1, 4)
2
>>> vote.index('maybe')
Traceback (most recent call last):
  File "<pyshell#74>", line 1, in <module>
    votes.index('maybe')
ValueError: tuple.index(x): x not in tuple

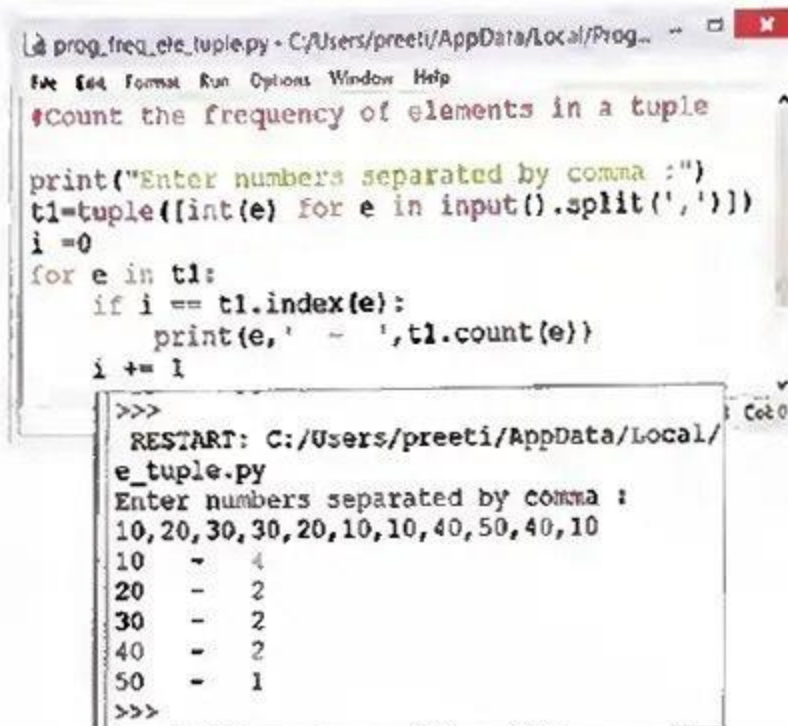
```

index() function generates an error when 'maybe' is entered as the argument as there is no such element present inside the tuple 'vote'.

Now, let us see an example based on the implementation of index() and count() functions of a tuple.

Practical Implementation-6

Program to count the frequency of elements in a tuple.



```
La prog_freq_ele_tuple.py - C:/Users/preeti/AppData/Local/Prog...
File Edit Format Run Options Window Help
#Count the frequency of elements in a tuple

print("Enter numbers separated by comma :")
t1=tuple([int(e) for e in input().split(',')])
i =0
for e in t1:
    if i == t1.index(e):
        print(e, ' ~ ', t1.count(e))
    i += 1

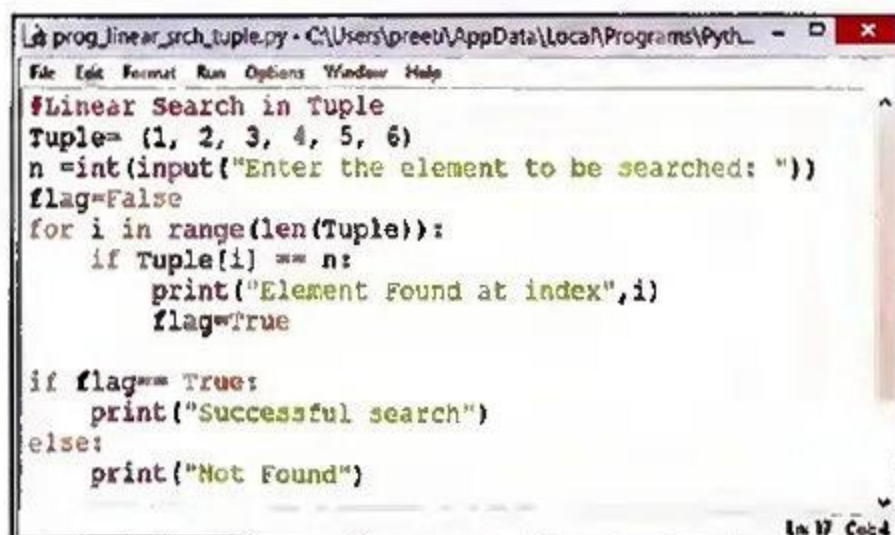
>>>
RESTART: C:/Users/preeti/AppData/Local/
e_tuple.py
Enter numbers separated by comma :
10,20,30,30,20,10,10,40,50,40,10
10 ~ 4
20 ~ 2
30 ~ 2
40 ~ 2
50 ~ 1
>>>
```

The above program counts the frequency of each element in a tuple. This is achieved by using two built-in functions available for tuple, i.e., count() and index(). First, the input for tuple elements is fetched from the user using list comprehension, where each element is accepted through input() and each element is separated using split() and comma as the separator. These individual elements separated are stored as list elements, which are further converted to a tuple by using tuple().

All the elements of tuple are compared iteratively with each and every element present in the tuple. Each occurrence of an element is counted using count(e) and for next iteration the index, i is incremented by 1 and hence the output is so obtained.

Practical Implementation-7

Program to perform linear search on a tuple of numbers.



```
La prog_linear_srch_tuple.py - C:/Users/preeti/AppData/Local/Programs/Pyth...
File Edit Format Run Options Window Help
#Linear Search in Tuple
Tuple= (1, 2, 3, 4, 5, 6)
n =int(input("Enter the element to be searched: "))
flag=False
for i in range(len(Tuple)):
    if Tuple[i] == n:
        print("Element Found at index",i)
        flag=True

if flag== True:
    print("Successful search")
else:
    print("Not Found")

Ln 17 Col 4
```

```

Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
RESTART: C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\prog_linear_search_tuple.py
Enter the element to be searched: 5
Element Found at index 4
Successful search
>>>

```

9.7 COMPARING TUPLES

You can compare two tuples by using comparison operators, i.e., `<`, `>`, `==`, `!=`, etc.

The comparison operator starts by comparing the first element from each sequence. If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered (even if they are really big).

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC
v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more informat
ion.
>>> (0, 1, 2) > (0, 3, 4)
False

>>> (0, 1, 2000000) < (0, 3, 4)
True

>>> tup1=(1,2,3)
>>> tup2=(1.0,2.0,3.0)
>>> tup1==tup2
True

```

While comparing,
int and float are
considered as same.

```

>>> tup1 = (1, 2, 3)
>>> tup2 = (10, 20, 30)
>>> tup1 != tup2
True

```

9.8 DELETING A TUPLE

The `del` statement is used to delete a tuple.

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46)
[MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more info
rmation.
>>> tup=(1,2,3)
>>> print(tup)
(1, 2, 3)
>>> del tup
>>> print(tup)
Traceback (most recent call last):
  File "<pyshe11#20>", line 1, in <module>
    print(tup)
NameError: name 'tup' is not defined

```

So, in a nutshell, tuples have the following features:

- You can't add elements to a tuple because of their immutable property. There's no `append()` or `extend()` method for tuples.
- You can't remove elements from a tuple, also because of their immutability. Tuples have no `remove()` or `pop()` method.
- You can find elements in a tuple since this doesn't change the tuple.
- You can also use the `in` operator to check if an element exists in the tuple.
- To change a tuple you can unpack it first, then change the value and pack it again.
- You can use `list()` and `tuple()` methods to change the values of a tuple.

Advantages of using tuples:

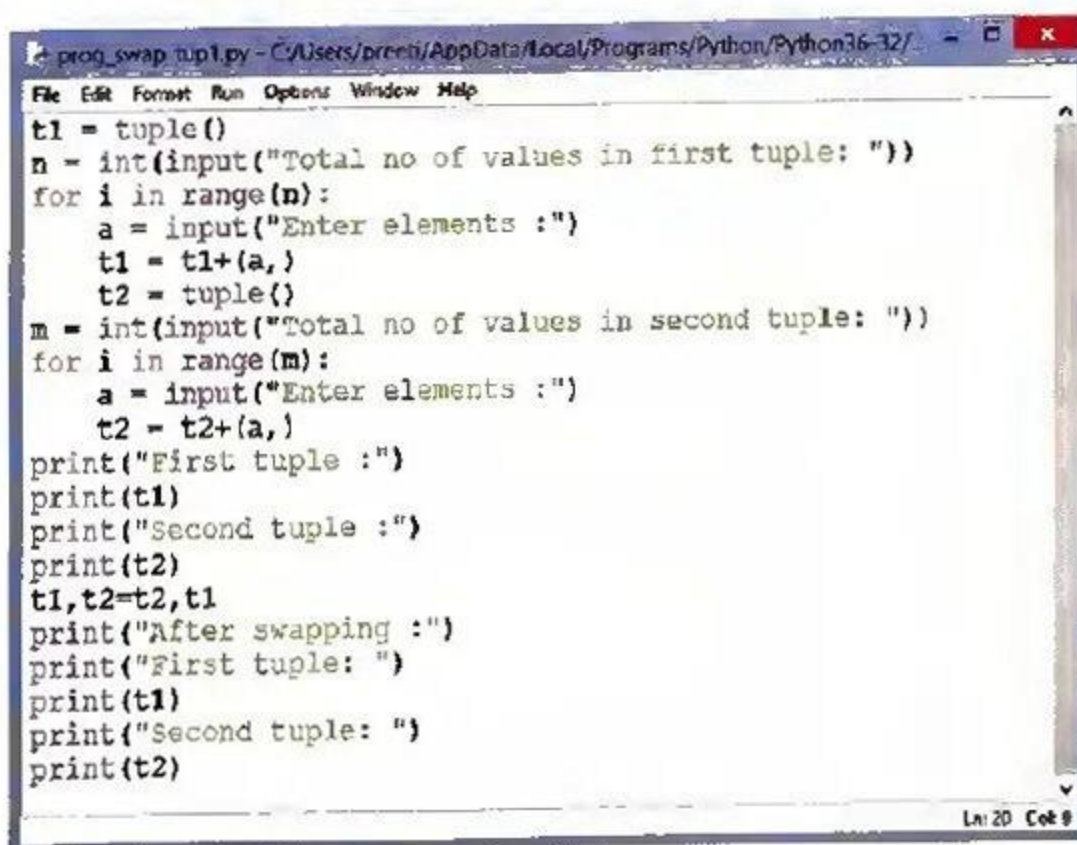
- Tuples use less memory whereas lists use more memory.
- We can use tuples in a dictionary as a key but it is not possible with lists.
- We can access element with an index in both tuples and lists.

Disadvantages of using tuples:

- We cannot add an element to a tuple but we can add an element to a list.
- We can't sort a tuple but in a list we can sort by calling "`list.sort()`" method.
- We can't remove an element in a tuple but in list we can remove an element.
- We can't replace an element in a tuple but we can replace in a list.

Practical Implementation-8

Write a Python program to input any two tuples and swap their values.



```

prog_swap_tuple.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/
File Edit Format Run Options Window Help
t1 = tuple()
n = int(input("Total no of values in first tuple: "))
for i in range(n):
    a = input("Enter elements :")
    t1 = t1+(a,)
    t2 = tuple()
m = int(input("Total no of values in second tuple: "))
for i in range(m):
    a = input("Enter elements :")
    t2 = t2+(a,)
print("First tuple :")
print(t1)
print("Second tuple :")
print(t2)
t1,t2=t2,t1
print("After swapping :")
print("First tuple: ")
print(t1)
print("Second tuple: ")
print(t2)
Ln: 20 Col: 9
  
```

```

Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900
32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/
prog_swap_tuple.py
Total no of values in first tuple: 4
Enter elements : 10
Enter elements : 20
Enter elements : 30
Enter elements : 40
Total no of values in second tuple: 3
Enter elements : 100
Enter elements : 200
Enter elements : 300
First tuple :
('10', '20', '30', '40')
Second tuple :
('100', '200', '300')
After swapping :
First tuple:
('100', '200', '300')
Second tuple:
('10', '20', '30', '40')
>>>

```

In Python, tuples are like lists and are declared by parentheses/round brackets. They are used to store immutable objects. As a result, they cannot be modified or changed throughout the program. In Python, there is a very powerful tuple assignment feature that assigns right-hand side of values to the left-hand side. In other words, it is called unpacking of a tuple of values into a variable. In packing, we put values into a new tuple while in unpacking we extract those values into a single variable.

Note: It must be ensured while unpacking tuple that the number of variables on the left-hand side should be equal to number of values in a given tuple.

For example,

```

>>> color=('red','green','yellow')
>>> (apple, orange, mango)=color
>>> print(apple)
red
>>> print(orange)
green
>>> print(mango)
yellow
>>>

```

```

tuple3.py - C:/Users/User2/AppData/Local/Programs/Python/Python39/tuple3.py (3.9.0)
File Edit Format Run Options Window Help
#Packing and unpacking tuple
tuple1=10,5,2      # Packing a tuple
print(tuple1)
x,y,z=tuple1      #Unpacking a tuple
print("x:",x)
print('y:',y)
print('z:',z)
print(x+y+z)
# the number of variables must be equal to the number of items of the tuple
x,y,z,p=tuple1

```

```

=== RESTART: C:/Users/User2/AppData/Local/Programs/Python/Python39/tuple3.py ===
(10, 5, 2)
x: 10
y 5
z 2
17
Traceback (most recent call last):
  File "C:/Users/User2/AppData/Local/Programs/Python/Python39/tuple3.py", line 1
    0, in <module>
      x,y,z,p=tuple1
ValueError: not enough values to unpack (expected 4, got 3)

```

In the above output, statement `(x, y, z, p = tuple1)` results in `ValueError` since there is a mismatch in the number of elements. Tuple holds three values which are being assigned to four variables, which is not possible and, hence, generates an error.

Thus, forming a tuple from individual values is called **packing** and creating individual values from a tuple's elements is called **unpacking**.

9.9 DICTIONARY IN PYTHON

You can use collections to create database-like structures using Python. Each collection type has a different purpose, and you use the various types in specific ways. The important idea to remember is that collections are simply another kind of sequence. Here, we are going to discuss a collection in Python which is known as **Dictionary**.

A telephone directory can be compared to a dictionary in Python. In a telephone directory, the subscriber's number gets stored according to his name and address. So, name becomes a key for reference and telephone number becomes the value to be searched for. Similarly, in Python too, we have a concept of key-value pair using dictionary.



Python dictionary is an unordered collection of items where each item is a **key-value** pair. We can also refer to a dictionary as a mapping between a set of keys/indices and a set of values.

```
{ "Python" : "Dictionaries" }
```

➤ Important features of Dictionaries are:

1. Each key map to a value. The association of a key and a value is called a key-value pair.
2. Each key is separated from its value by a colon (:), the items are separated by commas, and the entire dictionary is enclosed in curly braces {}.
3. Keys are unique within a dictionary while values may not be.
4. The value of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.
5. Dictionary is mutable. We can add new items or change the value of existing items.

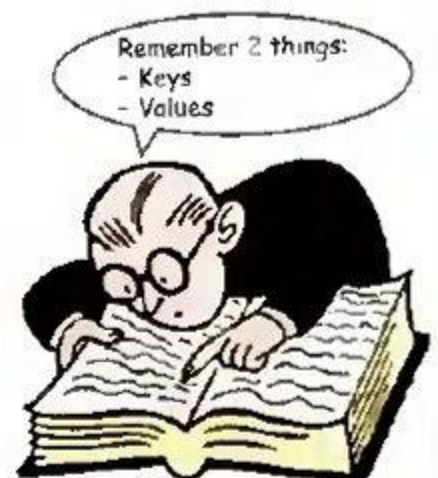
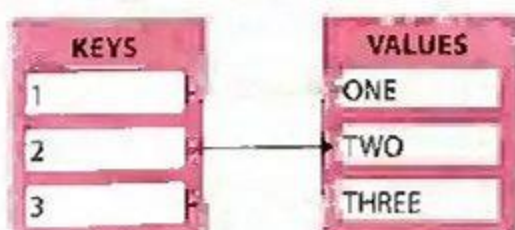


Fig. 9.8: Dictionary in Python



Creating a Dictionary

A dictionary can be created by placing items (i.e., key-value pair) inside curly braces {}, separated by comma.

Syntax:

```
<dictionary_name> = {'key1': 'value1', 'key2': 'value2',  
                      'key3': 'value3'...'keyn': 'valuen'}
```

A dictionary is extremely useful data storage construct for storing and retrieving all key value pairs, where each element is accessed (or indexed) by a unique key.

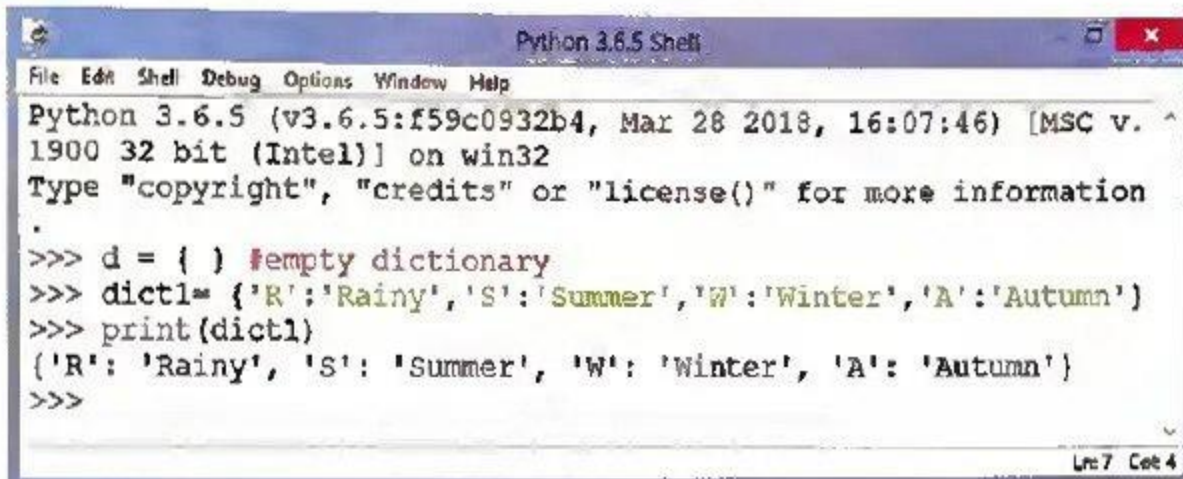
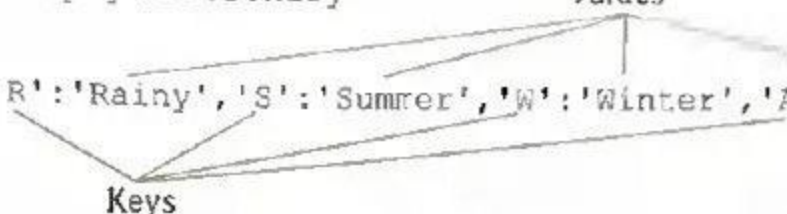
Examples of dictionary

```
>>> d={} # empty dictionary
```

Values

```
>>> dict1= {'R': 'Rainy', 'S': 'Summer', 'W': 'Winter', 'A': 'Autumn'}
```

Keys



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v. 1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information
>>> d = { } #empty dictionary
>>> dict1= {'R': 'Rainy', 'S': 'Summer', 'W': 'Winter', 'A': 'Autumn'}
>>> print(dict1)
{'R': 'Rainy', 'S': 'Summer', 'W': 'Winter', 'A': 'Autumn'}
>>>
```

In the second example, 'R', 'S', 'W' and 'A' are the keys and 'Rainy', 'Summer', 'Winter' and 'Autumn' are the values associated with the keys.

```
>>> d1={1: 'one', 2: 'two', 3: 'Three', 4: 'four'} # key as integer and values are strings
>>> d1={1: 10, 2: 20, 3: 30, 4: 40} # here both keys and values are integers
```



Key-value pairs

```
>>> d2={1: "one", "two": 2, (1,2,3): "numbers", 1.5: "float"}
```

```
>>> d2
```

```
{1: 'one', 'two': 2, (1, 2, 3): 'numbers', 1.5: 'float'}
```

Here, different data types are used to define keys of the dictionary like integer, string, tuples and float.

Note: Dictionary keys are case-sensitive, i.e., same name but different cases of keys will be treated distinctly.



9.10 METHODS TO CREATE A DICTIONARY

To create a dictionary, key-value pairs are separated by a comma and are enclosed in two curly braces {}. In key-value pair, each key is separated from its value by a colon (:).

Practical Implementation-9

To create an empty dictionary, by assigning two curly braces to a variable name:

```
>>> d1 = {}  
>>> type (d1)  
<class 'dict'>
```

Practical Implementation-10

To create a dictionary using string data type for both key-value pairs.

```
>>> d2={'input':'keyboard','output':'printer',  
      'language':'python','OS':'windows-8'}  
>>> print(d2)  
{'input': 'keyboard', 'output': 'printer', 'language': 'python', 'OS':  
'windows-8'}
```

Practical Implementation-11

To create a dictionary with built-in function dict(). The function dict() is used to create a new dictionary with no items.

```
>>> D=dict()  
>>> print(D)  
{}
```

{ } represent empty dictionary.

Practical Implementation-12

To create a copy of dictionary by passing a dictionary as an argument to dict() function.

```
>>> a={1:1,2:2}  
>>> b=dict(a)  
>>> b  
{1: 1, 2: 2}
```

Practical Implementation-13

To create a dictionary by passing nested list as an argument to dict() function.

```
a=[[1,"one"],[2,"two"],[3,"three"]]  
>>> c=dict(a)  
>>> c  
{1: 'one', 2: 'two', 3: 'three'}
```

practical Implementation-14

To add an item to the dictionary, we can use square brackets[] for accessing and initializing dictionary values.

For example,

```
>>> D = {}
>>> D[1] = "ONE"
>>> D[2] = "TWO"
>>> D[3] = "THREE"
>>> D[4] = "FOUR"
>>> print(D)
{1: 'ONE', 2: 'TWO', 3: 'THREE', 4: 'FOUR'}
```

Here, 1, 2, 3 and 4 are keys corresponding to values "ONE", "TWO", "THREE", "FOUR" respectively.

9.11 ACCESSING ELEMENTS IN A DICTIONARY

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. For example,

```
>>> dict1 = {'Name': 'Riya', 'Age': 7, 'Class': 'second'}
>>> print("dict1['Name']: ", dict1['Name'])
dict1['Name']: Riya
>>> print("dict1['Age']: ", dict1['Age'])
dict1['Age']: 7
```

Note: To access dictionary elements, you can use the square brackets along with the key to obtain its value.

If we attempt to access a data item with a key, which is not a part of the dictionary, we get an error as follows:

```
>>> dict1 = {'Name': 'Riya', 'Age': 7, 'class': 'second'}
>>> print("dict1['Aman']: ", dict1['Aman'])
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    print("dict ['Aman']: ", dict ['Aman'])
KeyError: 'Aman'
```

9.12 TRAVERSING A DICTIONARY

Traversing a dictionary means accessing each element of a dictionary. This can be done by using a for loop.

in operator using for loop:

Code:

```
>>> d={1:'one',2:'two',3:'three',4:'four'}
>>> for i in d:
    print(i,':', d[i])
1 : one
2 : two
3 : three
4 : four
```

Note: Traversing a dictionary means accessing each element of a dictionary. This can be done by using for loop.

Here, 'in' operator in for loop iterates each key of a dictionary.

Practical Implementation-15

Write a program to input total number of sections and stream name in 11th class and display all information on the output screen.

Code:

```
prgm_5.py - C:/Chapter 8/prgm_5.py (3.6.5)
File Edit Format Run Options Window Help
classxi=dict()
n=int(input("Enter total number of sections in xi class: "))
i=1
while i<=n:
    a=input("Enter Section: ")
    b=input("Enter stream name: ")
    classxi[a]=b
    i=i+1
print("Class",'\t',"Section",'\t',"Stream name")
for i in classxi:
    print("XI",'\t',i,'\t',classxi[i])
Ln: 12 Col: 0
```

Output

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Chapter 8/prgm_5.py =====
Enter total number of sections in xi class: 5
Enter Section: A
Enter stream name: SCIENCE WITH ECO
Enter Section: B
Enter stream name: SCIENCE WITH BIO
Enter Section: C
Enter stream name: SCIENCE WITH CS
Enter Section: D
Enter stream name: COMMERCE WITH MATHS
Enter Section: E
Enter stream name: COMMERCE WITH IP
Class      Section      Stream name
XI         A          SCIENCE WITH ECO
XI         B          SCIENCE WITH BIO
XI         C          SCIENCE WITH CS
XI         D          COMMERCE WITH MATHS
XI         E          COMMERCE WITH IP
Ln: 22 Col: 4
```

9.13 APPENDING VALUES TO A DICTIONARY

We can add new elements to the existing dictionary, extend it with single pair of values or join two dictionaries into one. If we want to add only one element to the dictionary, then we should use the following method.

Syntax:

```
Dictionary_name[key]=value
```

Note: We can add new elements to the existing dictionary, extend it with single pair of values or join two dictionaries into one.

For example,

```
>>> d1={'mon': 'monday', 'tue': 'tuesday', 'wed': 'wednesday'}
>>> d1['thu']='thursday'
>>> print(d1)
{'mon': 'monday', 'tue': 'tuesday', 'wed': 'wednesday', 'thu': 'thursday'}
```

9.14 UPDATING ELEMENTS IN A DICTIONARY

You can also update a dictionary by modifying existing key-value pair or by merging another dictionary with an existing one.

Syntax:

`<Dictionary>[<key>]=<value>`

If key is there in the dictionary, then it will update the value of that particular key in the dictionary.

For example,

```
>>> Dict = {'Teena': 18, 'Riya': 12, 'Alya': 22, 'Ravi': 25}
```

```
>>> Dict['Riya']=28
```

```
>>> print(Dict)
```

```
{'Teena': 18, 'Riya': 28, 'Alya': 22, 'Ravi': 25}
```

```
>>> Dict['Priya']=60
```

```
>>> print(Dict)
```

```
{'Teena': 18, 'Riya': 28, 'Alya': 22, 'Ravi': 25, 'Priya': 60}
```

If key is not there in the dictionary, then it will add key-value pair in the dictionary.

Note: While adding a value, if the key value already exists, the value gets updated, otherwise a new key with the value is added at the end of the dictionary.

Two dictionaries can be merged into one by using `update()` method. It merges the keys and values of one dictionary with another and overwrites values of the same key.

Syntax:

`Dic_name1.update(dic_name2)`

Using this, `dic_name2` is added to `Dic_name1`.

For example,

```
>>> d1={1:10,2:20,3:30}
```

```
>>> d2={4:40,5:50}
```

```
>>> d1.update(d2)
```

```
>>> print(d1)
```

```
{1: 10, 2: 20, 3: 30, 4: 40, 5: 50}
```

```
>>> print(d2)
```

```
{4: 40, 5: 50}
```

For example,

```
>>> d1={1: 10, 2:30, 3: 30, 5: 40, 6: 60}
```

```
>>> d2={1:10,2:20,3:30}
```

```
>>> d1.update(d2)
```

```
>>> print(d1)
```

```
{1: 10, 2: 20, 3: 30, 5: 40, 6: 60}
```

```
>>> print(d2)
```

```
{1: 10, 2: 20, 3: 30}
```

Value of key 2 is replaced with 20 in the dictionary d1.

9.15 REMOVING AN ITEM FROM DICTIONARY

We can remove an item from the existing dictionary by using `del` command or using `pop()` method;

1. using `del` command:

Syntax:

```
del dicname[key]
```

For example,

```
>>> A={"mon":"monday","tue":"tuesday","wed":"wednesday",  
      "thu":"thursday"}
```

```
>>> del A['wed']
```

```
>>> print(A)
```

```
{'mon': 'monday', 'tue': 'tuesday', 'thu': 'thursday'}
```

```
>>> del A['fri']
```

Traceback (most recent call last):

```
File "<pyshell#3>", line 1, in <module>  
    del A['fri']
```

KeyError: 'fri'

If you want to delete the entire dictionary, then give the dictionary name along with `del` keyword.

For example,

```
>>>del A
```

```
>>>A
```

Traceback (most recent call last):

```
File "<pyshell#31>", line 1, in <module>  
    A
```

NameError: name 'A' is not defined

The dictionary named A no longer exists in the memory.

2. using `pop()` method: `pop()` method will not only delete the item specified by the key from the dictionary but also return the deleted value.

Syntax:

```
Dictname.pop(key)
```

```
>>> D3={'mon':'Monday','tue':'tuesday','wed':'wednesday'}
```

```
>>> D3.pop('tue')
```

```
'Tuesday'
```

```
>>> print(D3)
```

```
{'mon': 'monday', 'wed': 'wednesday'}
```

Note: We can remove an item from the existing dictionary by using `del` command or using `pop()` method.

If particular key is not there, then it will result in `KeyError`.

It will return the deleted value.

9.16 'in' AND 'not in' MEMBERSHIP OPERATOR (in Dictionary)

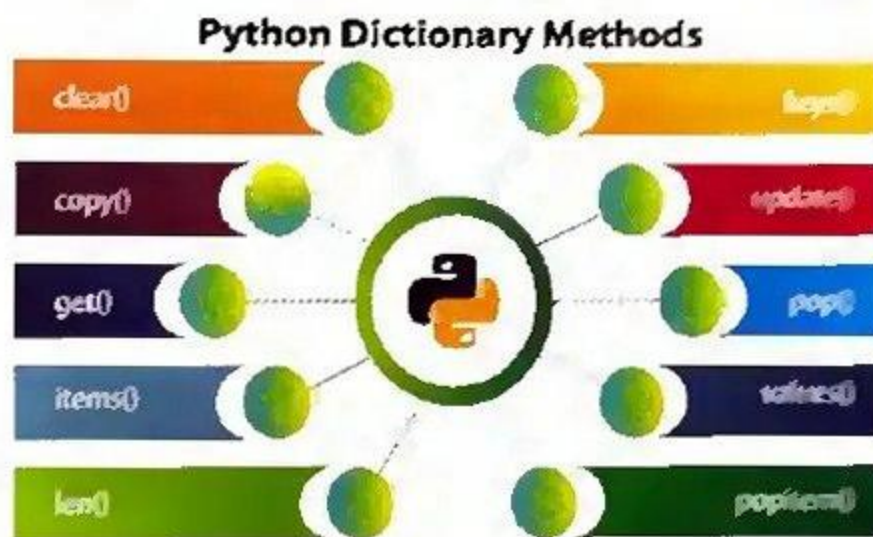
The 'in' operator checks whether a particular key is there in the dictionary. It returns True if specified key element appears in the dictionary, otherwise returns False.

```
>>> A={"mon":"monday","tue":"tuesday","wed":"wednesday",
      "thursday"}
>>> 'thu' in A
True
>>> 'fri' in A
False
>>> 'fri' not in A
True
```

CTM: Membership operation in, and functions like `min()`, `max()`, and `sum()` apply only to the keys of a dictionary.

9.17 COMMON DICTIONARY FUNCTIONS AND METHODS

Python provides you with a number of ways to manipulate the dictionary. Let's check out some of the important ones with examples.



len()

This method returns number of key-value pairs in the given dictionary.

Syntax:

```
len(d) # d is dictionary
```

returns number of key-value pairs in the dictionary.

```
>>> d1={1: 10, 2: 30, 3: 30, 5: 40, 6: 60}
>>> len(d1)
5
```

clear()

It removes all items from the particular dictionary.

Syntax:

```
d.clear() # d is dictionary
```

For example,

```
>>> D={1:'one',2:'two',3:'three',4:'four'}
>>> print(D)
{1: 'one', 2: 'two', 3: 'three', 4: 'four'}
>>> D.clear()
>>> print(D)
{}
```

get() method

The get() method returns a value for the given key. If key is not available, then returns default value None.

Syntax:

Following is the syntax for get() method:

```
dict.get(key, default = None)
```

key: This is the key to be searched in the dictionary.

default: This is the value to be returned in case the key does not exist.

This method returns a value for the given key. If key is not available, then returns default value None.

For example,

```
>>> D1={'sun':'Sunday','mon':'Monday','tue':'Tuesday','wed':'wednesday',
      'thu':'Thursday'}
>>> D1.get('wed')
'Wednesday'
>>> print(D1.get('fri'))
None
```



will return None as key does not exist

You can specify your own message also :

```
>>> D1.get('fri','never')
'never'
>>> D1.get('mon','never')
'Monday'
```

Here we have specified our own message but if the key exists, then it will return its value, not customized.

Note: items() returns a list of tuples having key-value pairs.

items()

It returns the content of dictionary as a list of tuples having key-value pairs.

Syntax:

```
D.items() #D dictionary
```

For example,

```
D={'sun':'Sunday','mon':'Monday','tue':'Tuesday','wed':'Wednesday','thu':'Thursday'}
D.items()
dict_items([('sun', 'Sunday'), ('mon', 'Monday'), ('tue', 'Tuesday'), ('wed', 'Wednesday'), ('thu', 'Thursday')])
```

Note It is called `items()` and because in python, items and dictionary values are written in the form of key-value pairs separated by

keys()

It returns a list of the key values in a dictionary.

Syntax:

```
D.keys() #D dictionary
```

For example,

```
>>> D={'sun':'Sunday','mon':'Monday','tue':'Tuesday','wed':'Wednesday','thu':'Thursday'}
>>> D.keys()
dict_keys(['sun', 'mon', 'tue', 'wed', 'thu'])
```

values()

It returns a list of values from key-value pairs in a dictionary.

Syntax:

```
D.values() #D dictionary
>>> D={'sun':'Sunday','mon':'Monday','tue':'Tuesday','wed':'Wednesday','thu':'Thursday'}
>>> D.values()
dict_values(['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday'])
```

copy()

We cannot copy a dictionary by using assignment (`=`) operator as it will create a reference to the same dictionary variable and modifications will also be reflected in both dictionaries. We can use built-in function `copy()` to create a new dictionary and modifications done on base dictionary will not be reflected.

Syntax:

```
Dict_new=Dict_old.copy()
```



For example,

```
>>> d1={"January":31,"February":28}
```

```
>>> d2=d1
```

Assigning a dictionary

```
>>> d1
```

```
{'January': 31, 'February': 28}
```

```
>>> d2
```

```
{'January': 31, 'February': 28}
```

```
>>> d2["March"]=31
```

```
>>> d2
```

```
{'January': 31, 'February': 28, 'March': 31}
```

Changes made to d2 is reflected in d1

```
>>> d1
```

```
{'January': 31, 'February': 28, 'March': 31}
```

```
>>> d3=d1.copy()
```

Copying a dictionary

```
>>> d3
```

```
{'January': 31, 'February': 28, 'March': 31}
```

```
>>> d1["April"]=30
```

Changes made in d1 is not reflected in d3

```
>>> d3
```

```
{'January': 31, 'February': 28, 'March': 31}
```

```
>>> d1
```

```
{'January': 31, 'February': 28, 'March': 31, 'April': 30}
```

Practical Implementation-16

WAP to store students' information like admission number, roll number, name and marks in a dictionary and display information on the basis of admission number.

Code:

```
prgm_6py - C:\Chapter 8\prgm_6py (3.4.5)
File Edit Format Run Options Window Help
SCL=dict()
i=1
flag=0
n=int(input("Enter number of entries:"))
while i<=n:
    Adm=input("\nEnter admission no of a student:")
    nm=input("Enter name of the student:")
    section=input("Enter class and section:")
    per=float(input("Enter percentage of a student:"))
    b=(nm,section,per)
    SCL[Adm]=b
    i=i+1
l=SCL.keys()
for i in l:
    print("\nAdmno- ", i, ":")
    z=SCL[i]
    print("Name\t","class\t","per\t")
    for j in z:
        print(j, end="\t")
Lve 21 Cak 0
```

Output:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Chapter 8/prgm_6.py
Enter number of entries:4

Enter admission no of a student:100
Enter name of the student:AUSHIM
Enter class and section:XIA
Enter percentage of a student:98

Enter admission no of a student:200
Enter name of the student:NISHANT
Enter class and section:XIB
Enter percentage of a student:97

Enter admission no of a student:300
Enter name of the student:RAM
Enter class and section:XIC
Enter percentage of a student:85

Enter admission no of a student:400
Enter name of the student:RIYA
Enter class and section:XID
Enter percentage of a student:78

Admno- 100 :
Name      class  per
AUSHIM    XIA    98.0
Admno- 200 :
Name      class  per
NISHANT   XIB    97.0
Admno- 300 :
Name      class  per
RAM       XIC    85.0
Admno- 400 :
Name      class  per
RIYA      XID    78.0
```

Practical Implementation-17

Write a program to enter names of employees and their salaries as input and store them in a dictionary.

```
Program to create a dictionary with names of the employees and their salary
num = int(input("Enter the number of employees whose data to be stored: "))
count = 1
employee = dict() #create an empty dictionary
while count <= num:
    name = input("Enter the name of the Employee: ")
    salary = int(input("Enter the salary: "))
    employee[name] = salary
    count += 1
print("\n\nEMPLOYEE_NAME\tSALARY")
for k in employee:
    print(k, '\t\t', employee[k])
```

```
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-6
t.py
Enter the number of employees whose data to be stored: 2
Enter the name of the Employee: Teena
Enter the salary: 45000
Enter the name of the Employee: Ritu
Enter the salary: 55000

EMPLOYEE_NAME    SALARY
Teena            45000
Ritu             55000
```



```
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32
pyl.py
D1 : {'Name': 'Radhika', 'DOB': '2002-03-11', 'Marks': '98'}
D2 : {'Name': 'Radhika', 'DOB': '2002-03-11', 'Marks': '98'}
Location of D1 43201520
Location of D2 45896496
>>>
```

popitem()

This method removes last item from dictionary and returns this deleted item.

Syntax:

D.popitem() #D dictionary

```
prog_pop_item.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37_ - x
File Edit Format Run Options Window Help
Student= {"Rollno":10,"Name":"Harsh","Stream":"Science"}
print(Student)
S =Student.popitem()
print(S,end="removed from")
print(Student)
```

```
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32,
m.py
{'Rollno': 10, 'Name': 'Harsh', 'Stream': 'Science'}
('stream', 'Science')removed from{'Rollno': 10, 'Name': 'Harsh'}
>>>
```

setdefault()

This method returns the value of the item with the specified key. If the key does not exist, it inserts the key with the specified value.

Syntax:

<Value>=<Dict>.setdefault(<key>,<default_value>)

The setdefault() function returns:

- Value of the key, if it is in the dictionary
- None, if key is not in the dictionary and default_value is not specified
- Default_value, if key is not in the dictionary and default_value is specified

```
prog_set_default1.py - C:/Users/preeti/AppData/Local/Programs/Pytho... - x
File Edit Format Run Options Window Help
D= {'Name': "Shaurya", 'Gender': 'Male'}
print(D)
D_Name= D.setdefault('Name',"Name not Available")
D_DOB= D.setdefault('DOB',"Date not Available")
D_Gender= D.setdefault('Gender')
D_Mobile= D.setdefault('Mobile')
print("Name :",D_Name)
print("DOB :",D_DOB)
print("Gender :",D_Gender)
print("Mobile :",D_Mobile)
```

```
>>>
RESTART: C:/Users/preeti/AppData/Local/
ault1.py
{'Name': 'Shaurya', 'Gender': 'Male'}
Name : Shaurya
DOB : Date not Available
Gender : Male
Mobile : None
>>>
```

max() and min()

The method max() returns key having maximum value. On the contrary, min() returns key having minimum value.

Syntax:

```
prog_max_min_dict.py - C:/Users/preeti/AppData/Local/Programs/Python...
File Edit Format Run Options Window Help
D={'Student1':80,'Student2':78,'Student3':76}
print("Highest Key with its value: ",max(D.items()))
print("Lowest Key with its value: ",min(D.items()))
print("Highest Key:",max(D))
print("Lowest Key:",min(D))
print("Highest Value:",max(D.values()))
print("Lowest Value:",min(D.values()))

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python...
dict.py
Highest Key with its value: ('Student3', 76)
Lowest Key with its value: ('Student1', 80)
Highest Key: Student3
Lowest Key: Student1
Highest Value: 80
Lowest Value: 76
>>>
```

sorted()

This method sorts the elements of a dictionary by its key or value.

```
prog_sorted_dict.py - C:/Users/preeti/AppData/Local/Programs/Python...
File Edit Format Run Options Window Help
D={'Student1':80,'Student2':78,'Student3':76}
L1=sorted(D)
print(L1)
L2=dict(sorted(D.items()))
print(L2)
L3=sorted(D.values())
print(L3)

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python...
dict.py
['Student1', 'Student2', 'Student3']
{'Student1': 80, 'Student2': 78, 'Student3': 76}
[76, 78, 80]
>>>
```



MEMORY BYTES

- A tuple is an immutable sequence of values which can be of any type and they are indexed by an integer.
- Creating a tuple is as simple as putting values separated by a comma. Tuples can be created using parentheses ().
- To create a tuple with single element, the final comma is necessary.
- Python provides various operators like '+', '*', 'in', 'not in', etc., which can be applied to tuples.
- Traversing a tuple means accessing each element of a tuple. This can be done by using looping statement either *for* or *while*.
- Python provides us with a number of tuple methods like len(), count(), any(), tuple(), max(), min(), mean(), sum(), sorted(), index(), etc.

- We can compare two tuples by using comparison operators, i.e., $<$, $>$, $=$, $!=$, etc.
- For tuple assignment, number of variables on the left side of assignment and the number of elements on the right side must match.
- **del statement** is used to delete a tuple.
- A Python dictionary is a mapping of unique keys to values. It is a collection of key-value pairs. Dictionaries are mutable which means they can be changed.
- In a dictionary, each key maps a value. The association of a key and a value is called a **key-value pair**.
- To create a dictionary, key-value pairs are separated by a comma and are enclosed in two curly braces $\{\}$. In key-value pair, each key is separated from its value by a colon $(:)$.
- Keys are unique and act as the index.
- Keys of a dictionary are of immutable type but values can be mutable.
- We can add new elements to the existing dictionary, extend it with single pair of values or join two dictionaries into one.
- You can also update a dictionary by modifying existing key-value pair or by merging another dictionary with an existing one.
- Python provides you with a number of dictionary methods like `len()`, `pop()`, `items()`, `keys()`, `values()`, `get()`, etc.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) Elements in a tuple can be of type.
- (b) Tuples are indexed by an
- (c) Tuples are, you cannot update or edit the elements.
- (d) are mutable, ordered collection with element in the form of key.
- (e) function is used to convert a sequence data type into tuple.
- (f) Like lists, dictionaries are which means they can be changed.
- (g) A Python dictionary is a of unique keys to values.
- (h) The association of a key and a value is called a
- (i) To create a dictionary, key-value pairs are separated by
- (j) In key-value pair, each key is separated from its value by a
- (k) function returns the keys of the dictionary.
- (l) The statement removes a dictionary object along with its items.
- (m) function will delete the last inserted key-value pair from the dictionary.

2. State whether the following statements are True or False.

- (a) An element in a dictionary is a combination of key-value pair.
- (b) Internally, dictionaries are indexed on the basis of values.
- (c) We can repeat a key in dictionary.
- (d) In dictionary, two unique keys can have the same values.
- (e) A tuple can be a key if it contains immutable elements.
- (f) `clear()` function is used to delete a dictionary.
- (g) A tuple is a mutable data type.
- (h) Lists can be used as keys in a dictionary.
- (i) Updating an element in a dictionary at runtime is possible.
- (j) We can create immutable dictionaries in Python.
- (k) A tuple can store elements of different data types.
- (l) A tuple cannot store list as an element.
- (m) Creating individual values from a tuple's elements is called packing.
- (n) We cannot delete a dictionary once created.



3. Multiple Choice Questions (MCQs)

- (a) Which of the statement(s) is/are correct?
- (i) Python Dictionary is an ordered collection of items
 - (ii) Python Dictionary is a mapping of unique keys to values
 - (iii) Dictionary is mutable
 - (iv) All of these
- (b) If `tup = (20,30,40,50)`, which of the following is incorrect?
- (i) `print(tup[3])`
 - (ii) `tup[2] = 56`
 - (iii) `print(max(tup))`
 - (iv) `print(len(tup))`
- (c) Consider two tuples given below:
- ```
tup1 = (1,2,4,3)
tup2 = (1,2,3,4)
```
- What will the following statement print. `print(tup1 < tup2)`
- (i) True
  - (ii) False
  - (iii) Error
  - (iv) None of these
- (d) Which function returns the number of elements in the tuple?
- (i) `len()`
  - (ii) `max()`
  - (iii) `min()`
  - (iv) `count()`
- (e) To create an empty dictionary, we use
- (i) `d1={}`
  - (ii) `d1=[]`
  - (iii) `d1=()`
  - (iv) `d1=={}`
- (f) In a dictionary, the elements are accessed through
- (i) Key
  - (ii) Value
  - (iii) Index
  - (iv) None of these
- (g) Keys of a dictionary must be
- (i) Similar
  - (ii) Unique
  - (iii) Can be similar or unique
  - (iv) All of these
- (h) To create new dictionary with no items:
- (i) Dict
  - (ii) `dict()`
  - (iii) `d1=[ ]`
  - (iv) None of these
- (i) Which function is used to return a value for the given key?
- (i) `len()`
  - (ii) `get()`
  - (iii) `keys()`
  - (iv) None of these
- (j) Which function is used to remove all items from a particular dictionary?
- (i) `clear()`
  - (ii) `pop()`
  - (iii) `delete()`
  - (iv) `rem()`
- (k) Which one of the following is correct to insert a single element in a tuple?
- (i) `T=4`
  - (ii) `T=(4)`
  - (iii) `T=(4,)`
  - (iv) `T=[4,]`
- (l) Which of the following functions will return key-value pairs of the dictionary?
- (i) `key()`
  - (ii) `values()`
  - (iii) `items()`
  - (iv) `get()`
- (m) Which of the following will delete key-value pair for key = "Red" from a dictionary D1?
- (i) `delete D1{"Red"}`
  - (ii) `del.D1{"Red"}`
  - (iii) `del D1["Red"]`
  - (iv) `del D1`
- (n) What will be the output?
- ```
D1={"Rahul":56,"Virat":99}
print("virat" in D1)
```
- (i) True
 - (ii) False
 - (iii) No output
 - (iv) Error

SOLVED QUESTIONS

1. What is a tuple?

Ans. A tuple is an immutable sequence of values which can be of any type and they are indexed by an integer.

2. What are the differences between lists and tuples?

Ans. The differences between tuples and lists are that tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

3. What is the similarity between strings and tuples?

Ans. Tuples are immutable like strings, you cannot update or edit a tuple. To change it, you have to create a new one just like strings.

4. Tuples can be used as keys in Python dictionaries. Explain.

Ans. Yes, tuples can be used as keys in Python dictionaries as keys of dictionary cannot be modified and tuples are immutable by default and hence can be used.

5. Tuple is an ordered immutable sequence of objects. Justify your answer.

Ans. Tuple is an ordered sequence of objects as each element is accessed by its index and it is immutable as you cannot change the values in place. You cannot update or edit the tuple.

6. What is a dictionary?

Ans. A Python dictionary is a mapping of unique keys to values. It is a collection of key-value pairs. Dictionaries are mutable which means they can be changed.

7. What are the differences between lists and dictionary?

Ans. The differences between dictionary and lists are that in lists, elements are enclosed within square brackets, whereas in dictionary, key-value pairs are enclosed within curly braces. In dictionary, index value can be of any data type and is called key, but in list index value is an integer.

8. Can list be used as a key to dictionary? Justify your answer.

Ans. Lists cannot be used as key to dictionary as they are mutable.

9. Write different ways of creating tuples with one item and print the same.

Ans. (a) `tup1=5,`
 `print(tup1)`
 (b) `tup1=(5,)`
 `print(tup1)`
 (c) `tup1=tuple((10,))`
 `print(tup1)`

10. Write a statement to add a single item in a tuple.

Ans. `>>> t=(10,20,30,40)`
 `>>> t=t+(60,)`

11. Write different ways of creating tuples with the following numbers:

5,10,15,20

Ans. (a) `t1=(5,10,15,20)` (b) `t1=5,10,15,20`
 (c) `t1=5,10,15,20,` (d) `t1=tuple((5,10,15,20))`
 (e) `t1=tuple([5,10,15,20])`

12. Following are the statements for creating tuples. Find the errors in the statements and rewrite the same after correcting them.

(a) `tup1=1,6,a,8,` (b) `tup2=(10)`
(c) `tup1=((0,1,2,3) ['my', 'book'])` (d) `tup1=((0,1,2,3) (4,5,6))`
(e) `tup1=('a','b','c' [1,2,3])`
ns. (a) `tup1=1,6,'a',8,` (b) `tup2=(10,)`
(c) `tup1=((0,1,2,3), ['my', 'book'])` (d) `tup1=((0,1,2,3), (4,5,6))`
(e) `tup1=('a','b','c', [1,2,3])`

13. Consider the following code and find out the error:

(a) `tup1=(1,'school',6,7)`
 `print(max(tup1))`
(b) `t=(1,'s','h',5,'p')`
 `t[1]='o'`
(c) `T1=(7)`
 `T2=(1,2,3)`
 `T3=T1+T2`
(d) `T1=(1,2,3)`
 `T2=(4,5,6)`
 `T3=(8,9)`
 `a,b,c = T1,T2`

- (e) `T1 = (1, 2, 3)`
`T4 = T1 * 2`
- (f) `tuple1 = (1, 2, 3, 4)`
`tuple1 = tuple1 + 10`
`print(tuple1)`
- (g) `tuple1 = ("5", 3)`
`print(tuple1)`
- (h) `cl = (1, 2, 3, 4)`
`a, b, c = cl`

- Ans.** (a) `max()` will work only if elements in a tuple are of the same data type.
 (b) Tuples are immutable, we cannot change the value of any element.
 (c) '+' operator joins two tuples, here (7) is 'int', so will give error.
 (d) For tuple assignment, left-hand side and right-hand side elements should be the same. T3 is not there on right-hand side.
 (e) '*' operator requires a tuple and an integer as operands.
 (f) The '+' operator simply performs a concatenation with tuples. Here 10 is an integer.
 (g) '*' operator requires a tuple and an integer as operands. Here '3' is a string.
 (h) For unpacking a tuple, number of variables on the left side of assignment and the number of elements in the tuple must match.

14. Find errors, underline them and rewrite the same after correcting the following code:

```
d1 = dict[]
i = 1
n = input("Enter number of entries:")
while i <= n:
    a = input("Enter name:")
    b = input("Enter age:")
    d1(a) = b
    i = i + 1
l = d1.key[]
for i in l:
    print(i, '\t', 'd1[i]')
```

Ans. `d1 = dict()`
`i = 1`
`n = int(input("Enter number of entries:"))`
`while i <= n:`
 `a = input("Enter name:")`
 `b = input("Enter age:")`
 `d1[a] = b`
 `i = i + 1`
`l = d1.keys()`
`for i in l:`
 `print(i, '\t', d1[i])`

15. Write the output of the following codes:

- (a) `>>> d1={1:10,2:20,3:30,4:40,5:50}`
`>>> d1.items()`
`>>> d1.keys()`
`>>> d1.values()`
- (b) `d1={1:10,2:20,3:30,4:40}`
`d2={5:50,6:60,7:70}`
`d1.update(d2)`
`print(d1)`



```
(c) d1={1:10,2:20,3:30,4:40,5:50,6:60,7:70}
    del d1[3]
    print(d1)
(d) d1={1:10,2:20,3:30,4:40,5:50,6:60,7:70}
    d1.pop(5)
    print(d1)
(e) d1={1:10,2:20,3:30,4:40,5:50,6:60,7:70}
    len(d1)
    d1.get(6,60)
    d1[6]=65
    d1[8]=35
    print(d1)
    d1.clear()
    print(d1)
```

Ans. (a) dict_items[(1, 10), (2, 20), (3, 30), (4, 40), (5, 50)]
 dict_keys[1, 2, 3, 4, 5]
 dict_values[10, 20, 30, 40, 50]
 (b) {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60, 7: 70}
 (c) {1: 10, 2: 20, 4: 40, 5: 50, 6: 60, 7: 70}
 (d) {5: 50}
 {1: 10, 2: 20, 3: 30, 4: 40, 6: 60, 7: 70}
 (e) 7
 60
 {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 65, 7: 70, 8: 35}
 {}

16. Write a Python program to display all the elements of the following tuple except 'd'.

```
t1=('w','o','r','d','s')
```

Ans. t1=('w','o','r','d','s')

```
t1=t1[0:3]+t1[-1:]
```

```
print(t1)
```

Output:

```
('w', 'o', 'r', 's')
```

17. Write a Python program to remove an element '3' from the following tuple:

```
t1=(1,2,3,4,5)
```

Ans. t1 = (1,2,3,4,5)

```
L = list(t1)
```

```
L.remove(3)
```

```
t1 = tuple(L)
```

```
print(t1)
```

Output:

```
(1, 2, 4, 5)
```

Alternatively, using pop() method:

with pop() we have to give index of element 3 as shown:

```
t1 = (1,2,3,4,5)
```

```
L = List(t1)
```

L.pop(2) #2 is the index value for element 3

```
t1 = tuple(L)
```

```
print(t1)
```

Output:

```
(1, 2, 4, 5)
```

18. t1=(10,20,"book",30,9.5,"item",[12,13],(3,4),30,5,30)

Consider the above tuple 't1' and answer the following questions:

(a) len(t1)

(b) t1[:6]

(c) t1[-8:-4]

(d) t1[5:]

(e) t1.index(20)

(f) t1.index(30)

(g) t1.index(30,7,10)

(h) t1.count(30)

(i) t1[:1]*5

(j) any(t1)

- Ans. (a) 11 (b) (10, 20, 'book', 30, 0.5, 'item')
 (c) (30, 0.5, 'item', [12, 13]) (d) ('item', [1, 13], (3, 4), 30, 5, 30)
 (e) 1 (f) 3
 (g) 8 (h) 3
 (i) (10, 10, 10, 10, 10) (j) True

19. T1 = ("TEA POT", [1, 2, '3'], 'S', (3, 4, 6), "book", 10)

Based on tuple T1 above, write the output for the following expressions:

- (a) T1[1][1] (b) 10 in T1
 (c) 50 not in T1 (d) T1[4][3]
 (e) T1[1][2]^3 (f) T1[-5: 3]+T1[3]
 (g) T1[0]-T1[2] (h) T1[4:4]

- Ans. (a) 2 (b) True
 (c) True (d) 'k'
 (e) '333' (f) ([1, 2, '3'], 'S', 3, 4, 6)
 (g) 'TEA POTS' (h) ()

20. Write the output of the following statements:

- (a) >>> tuple("book") (b) >>> tuple([1, 2, 4])
 (c) >>> ("Riya",)*3 (d) >>> max(10, 12, 73, 34)
 (e) >>> min(40, 20, 100, 60)

- Ans. (a) ('b', 'o', 'o', 'k') (b) (1, 2, 4)
 (c) ('Riya', 'Riya', 'Riya') (d) 73
 (e) 20

21. Compare the tuples and write the output:

- (a) >>> tup1=(1,2,3)
 >>> tup2=(1.0,2.0,3.0)
 >>> tup1==tup2
 (b) >>> tup1=('FARIDA','VAIBHAV','MINI','RAGHAV')
 >>> tup2=('Ravi','Mala','AMAN','SAKSHI')
 >>> tup1<tup2
 (c) >>> tup1=(10,20,30,40,50)
 >>> tup2=(4,5,10,20,30,40,50)
 >>> tup2>tup1
 (d) >>> tup1!=tup2

- Ans. (a) True (b) True
 (c) False (d) True

22. t1=(1,2,3,4)
 t2=(10,20,30,40)
 t3=(100,200,300,400)

Write the output of the following:

- (a) t1,t2=t2,t1
 print(t1)
 print(t2)
 (b) t1,t3=t3,t1
 print(t1)
 print(t3)
 (c) t3,t2,t1=t1,t2,t3
 print(t1)
 print(t2)
 print(t3)

Ans. (a) (10, 20, 30, 40)
(1, 2, 3, 4)

(b) (100, 200, 300, 400)
(10, 20, 30, 40)

(c) (100, 200, 300, 400)
(10, 20, 30, 40)
(1, 2, 3, 4)

23. Neeti has created a dictionary of month name with number of days in Python. You have to help her in completing the program:

```
Days={"January":31,"Feburary":28,"March":31,"April":30,"May":31,  
      "June":30,"July":31,"August":31,"September":30,"October":30,  
      "November":30,"December":31}
```

#Display months having 31 days

```
for Key,Value in _____: #line1
```

```
    if _____: #line 2
```

```
        print(Key)
```

```
        #display Months and days starting with "A" or "M"
```

```
        for Key in Days.keys():
```

```
            if _____: #line 3
```

```
                print(_____) #line 4
```

```
        #change number of days in February as 29
```

```
        _____ #line 5
```

```
        print(Days)
```

(a) The function to get both keys and values.

(b) The condition to check if number of days 31

(c) The display month starts with "A" or "M"

(d) Display month and days

(e) Change number of days as 29 for the month of February

Ans. (a) Days.items()

(b) if Value ==31:

(c) if Key in "AaMm":

(d) print(Key,":",Value)

(e) Days["February"]=29

24. Write a Python program to input 'n' names and phone numbers to store it in a dictionary and print the phone number of a particular name.

```
phone=dict()
```

```
i = 1
```

```
n=int(input("Enter number of entries:"))
```

```
while i<=n:
```

```
    a=input("Enter Name:")
```

```
    b=input("Enter Phone no:")
```

```
    phone[a]=b
```

```
    i = i+1
```

```
l = phone.keys()
```

```
x=input("Enter name to be searched:")
```

```
for i in l:
```

```
    if i==x:
```

```
        print (x," : phone no is:",phone[i])
```

```
        break
```

```
else:
```

```
    print(x, " does not exist")
```



Ans. Output

```
Enter number to be searched: 1
Enter Name: Aman
Enter Phone number: 9876543210
Enter Name: Arun
Enter Phone number: 1234567890
Enter Name: Hriday
Enter Phone number: 1234567890
Enter Name: Tanya
Enter Phone number: 9876543210
Enter name to be searched: Aman
Aman : phone no. is: 1111
```

25. Write a Python program to input names of 'n' employees and their salary details like basic salary, house rent and conveyance allowance. Calculate total salary of each employee and display

Code:

```

d1=dict()
i= 1
n=int(input("Enter number of entries:"))
while i<=n:
    Nm=input("\nEnter name of the employee: ")
    basic=int(input("Enter Basic salary: "))
    hra=int(input("Enter house rent allowance: "))
    ca=int(input("Enter conveyance allowance: "))
    d1[Nm]=[basic,hra,ca]
    i = i+1
l = d1.keys()
print("\nName",'\t\t',"Net Salary")
for i in l:
    salary = 0
    z= d1[i]
    for j in z:
        salary=salary+j
    print(i,'\t\t',salary)

```

Note: li

Note: list is stored in dictionary as value.

Ans. Output:

```
Enter number of entries:4
Enter name of the employee: Nishant
Enter Basic salary: 100000
Enter house rent allowance: 30000
Enter conveyance allowance: 5000
Enter name of the employee: Ram
Enter Basic salary: 50000
Enter house rent allowance: 15000
Enter conveyance allowance: 5000
Enter name of the employee: Aushim
Enter Basic salary: 200000
Enter house rent allowance: 50000
Enter conveyance allowance: 10000
Enter name of the employee: Riya
Enter Basic salary: 25000
Enter house rent allowance: 2000
Enter conveyance allowance: 1000
```

Name	Net Salary
Nishant	15000
Ram	70000
Asham	260000
Kiya	28000

26. Write a Python program to input names of 'n' countries and their capital and currency, store it in a dictionary and display in tabular form. Also search and display for a particular country.

Code:

```
dl=dict()
i=1
n=int(input("Enter number of entries:"))
while i<=n:
    c=input("Enter Country:")
    cap=input("Enter Capital:")
    curr=input("Enter currency of country:")
    dl[c]=(cap,curr)
    i=i+1
l=dl.keys()
print("\nCountry\t\t", "Capital\t\t", "Currency")
for i in l:
    z=dl[i]
    print('\n',i,'\t\t',end="")
    for j in z:
        print(j,'\t\t',end="\t\t")
# searching
x=input("\nEnter Country to be searched: ")
for i in l:
    if i==x:
        print("\nCountry\t\t", "Capital\t\t", "Currency\t\t")
        z=dl[i]
        print('\n',i,'\t\t',end="")
        for j in z:
            print(j,'\t\t',end="\t\t")
        break
```

Note: Tuple is stored in dictionary as value.

Ans. Output:

```
Enter number of entries:5
Enter Country:Austria
Enter Capital:Vienna
Enter currency of country:Euro
Enter Country:China
Enter Capital:Beijing
Enter currency of country:Yuan
Enter Country:India
Enter Capital:New Delhi
Enter currency of country:Indian Rupee
Enter Country:France
Enter Capital:Paris
Enter currency of country:Euro
Enter Country:United Kingdom
Enter Capital:London
Enter currency of country:Pound

Country      Capital      Currency
Austria      Vienna      Euro
China        Beijing     Yuan
India        New Delhi   Indian Rupee
France       Paris       Euro
United Kingdom London      Pound

Enter Country to be searched: India
Country      Capital      Currency
India        New Delhi    Indian Rupee
```



27. Write a program to convert a number entered by the user into its corresponding number in words.
For example, if the input is 985, then the output should be 'Nine Eight Five'.

Ans. #Program to convert number into corresponding number in words
#numberNames is a dictionary of digits and corresponding number
numberNames = {0: 'Zero', 1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: 'Five', 6: 'Six', 7: 'Seven', 8: 'Eight', 9: 'Nine'}
num = input("Enter any number: ") #number is stored as string
result = ''
for ch in num:
 key = int(ch) #converts character to integer
 value = numberNames[key]
 result = result + ' ' + value
print("The number is:", num)
print("The numberName is:", result)

28. Write a program to accept and store Employee name as a key and his salary, allowance and deduction as value in a dictionary Dict for 10 employees. Display the following Menu:

- Display Total Salary
 - Display Total Allowance and Deductions
 - Search an Employee
- When user selects option 1, display a report having Name, Salary, Allowance, Deduction, Gross salary (Salary + allowance) and Net Salary (Gross salary – Deductions)
 - When user selects option 2, find total allowance and deductions of all 10 employees.
 - When user selects option 3, accept name of the employee and search in the dictionary.

Ans. D={}
for a in range(5):
 name=input("Enter name:")
 salary=float(input("Enter salary:"))
 allowance=float(input("enter allowance:"))
 deduction=float(input("enter deduction:"))
 D[name]=[salary,allowance,deduction]
ch=0
while True:
 print("""
 1. Display the total salary
 2. Display total allowance and deduction
 3. search an employee
 4. exit""")
 ch=int(input("Enter your choice:"))
 if ch==1:
 for Key,value in D.items():
 gross=value[0]+value[1]
 net=gross-value[2]
 print(Key,gross,net)
 elif ch==2:
 total_allowance=0
 total_deduction=0
 for Key,value in D.items():
 total_allowance+=value[1]
 total_deduction+=value[2]
 print("Total allowance:",total_allowance)
 print("Total deduction:",total_deduction)

```

elif ch==3:
    n=input("Enter employee name to search:")
    for Key,value in D.items():
        if Key==n:
            print(Key,value[0],value[1],value[2])
        else:
            print("Not found")
elif ch==4:
    break
else:
    print("Wrong choice")

```

UNSOLVED QUESTIONS

1. What are the advantages of using tuples?
2. What are the similarities and differences between tuples and lists?
3. What is a key-value pair in dictionary?
4. What are the differences between tuple and dictionary?
5. What are the differences between strings and tuples?
6. Can we always have same type of values in a tuple? State True or False with justification.
7. What is nested tuple? Explain with example.
8. Write the code to create an empty tuple.
9. Write a Python program to create a tuple with different data types and display.
10. What is the difference between "book" and ('book',)?
11. Write a Python program to combine first 3 elements and last 3 elements from a tuple.

```
T1=('a',1,2,3,4,'b','c','book',10)
```

Output should be

```
('a',1,2,'c','book',10)
```

12. Find errors and rewrite the same after correcting the following code:

- (a) d1={1:10,2.5:20,3:30,4:40,5:50,6:60,7,70}
- (b) d1(9)=90
- (c) del d1(2)
- (d) pop d1[4]
- (e) d1.item()
- (f) d1.key()
- (g) d1.value()
- (h) d1.gets(4,80)
- (i) d1.len()
- (j) d1.clears()

13. Suppose

```
d1={1:'one',2:'two',3:'three',4:'four'}
```

```
d2={5:'five',6:'six'}
```

Write the output of the following code:

- (a) >>> d1.items()
>>> d1.keys()
>>> d1.values()
>>> d1.update(d2)
>>> len(d1)

```
(b) >>> del d1[3]
>>> print(d1)
>>> d1.pop(4)
>>> print(d1)
>>> d1[8]='eight'
>>> print(d1)
>>> d1.clear()
>>> print(d1)
```

14. Consider the following code and find out the error:

```
(a) tup1=(10,20,30)
    tup2= tup1*(3,)
(b) tup1=('a', 'b', 'c')
    tup1=tup1+'d'
(c) tup1=("Riya",)*'4'
(d) t1=(10,20,30,40)
    X,Y,Z=t1
(e) tup1=('A','school',10.5,70)
    print max(tup1)
(f) t=(10,20,30,40)
    t[1]='o'
(g) T1=('A')
    T2=('B','C',3)
    T3=T1+T2
(h) T1=(10,20,30,40)
    T2=(40,50,60)
    T1,T2,T3=T1,T2
```

15. `t1=(100,200,"Global",3,3.5,"Exam",[1,2],(30,40),(3,5,3))`

Consider the above tuple 't1' and answer the following questions:

(a) <code>len(t1)</code>	(b) <code>20 not in t1</code>
(c) <code>t1[-8:-4]</code>	(d) <code>t1[5:]</code>
(e) <code>t1.index(5)</code>	(f) <code>t1.index(3,6)</code>
(g) <code>10 in t1</code>	(h) <code>t1.count(3)</code>
(i) <code>any(t1)</code>	(j) <code>t1[2]*2</code>

16. Write the output of the following statements:

```
(a) >>> tuple([10,20,40])
(b) >>> ("Tea",)*5
(c) >>> tuple("Item")
```

17. Consider two tuples t1 & t2 given below:

```
t1=(100,200,300)
t2=(10,20,30,40)
```

Write the output of the following statements:

```
(a) t1,t2=t2,t1
    print(t1)
    print(t2)
(b) print(t1!=t2)
(c) print(t1<t2)
```

18. WAP to accept values from a user. Add a tuple to it and display its elements one by one. Also display its maximum and minimum value.
19. Write a Python program to input any values for two tuples. Print it, interchange it, and then compare them.
20. Write a Python program to input 'n' classes and names of their class teachers to store it in a dictionary and display the same. Also accept a particular class from the user and display the name of the class teacher of that class.
21. Write a Python program to store student names and their percentage in a dictionary, delete a particular student's name from the dictionary. Also display dictionary after deletion.
22. Write a Python program to input names of 'n' customers and their details like items bought, cost and phone number. Store it in a dictionary and display all the details in a tabular form.

CASE-BASED/SOURCE-BASED INTEGRATED QUESTIONS

1. Rajmandir Basket is a department store with a wide range of merchandise including groceries, fruits, vegetables and household accessories. It aims to develop a complete software for Billing, Inventory and Accounts management and to provide consumers a user-friendly interface to view item codes and prices of various items.

Write a program in Python that repeatedly asks the user to enter product names and prices. Store all of them in a dictionary whose keys are product names and values are prices. Also write a code to search an item from the dictionary.

Ans.

```

prog_departmental_store_dict1.py - C:\Users\preeti\AppData\Local\Programs\Python\Py...
File Edit Format Run Options Window Help
#program to store the details of groceries and their prices
dict={}
ch='y'
while ch=='y' or ch=='Y':
    name=input("Enter the product name:")
    price=eval(input("Enter the product price :"))
    dict[name]=price
    ch=input("Do you want to add more items (Y/N)")
print(dict)
prod_name=input("Enter the product to be searched :")
for i in dict:
    if i == prod_name:
        print("Product found",i,"with price as:",dict[i])

```

```

>>>
RESTART: C:\Users\preeti\AppData\Local\Programs
ental_store_dict1.py
Enter the product name: Sugar
Enter the product price : 60
Do you want to add more items (Y/N) y
Enter the product name: Rice
Enter the product price : 80
Do you want to add more items (Y/N) y
Enter the product name: Pulses
Enter the product price : 100
Do you want to add more items (Y/N) n
{'Sugar': 60, 'Rice': 80, 'Pulses': 100}
Enter the product to be searched : Rice
Product found Rice with price as: 80
>>>

```

2. The above shopping destination for groceries, Rajmandir, stores mobile numbers of its customers. Mobile numbers are to be stored in the form of a dictionary. Write a Python program to input 'n' names and mobile numbers to store them in a dictionary and to input any name and to print mobile number of that particular customer.

Ans.

```
C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\prog_out_
File Edit Format Run View Tools Windows Help
#program to store the details of customer names and mobile numbers
dict={}
ch=''
while ch=='y' or ch=='Y':
    custname=input("Enter the customer name:")
    phone=input("Enter the customer mobile number:")
    dict[custname]=phone
    ch=input("Do you want to add more customer details (Y/N):")
print(dict)
cust_name=input("Enter name of the customer to be searched for:")
for i in dict:
    if i==cust_name:
        print("Customer",cust_name,"has mobile number as:",dict[i])

>>>
RESTART: C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\prog_out_
_dict2.py
Enter the customer name:Adish
Enter the customer mobile number :98766544232
Do you want to add more customer details (Y/N)y
Enter the customer name:Teena
Enter the customer mobile number :9987654390
Do you want to add more customer details (Y/N)y
Enter the customer name:Ajay
Enter the customer mobile number :9807765488
Do you want to add more customer details (Y/N)n
{'Adish': '98766544232', 'Teena': '9987654390', 'Ajay': '9807765488'}
Enter name of the customer to be searched for :Ajay
Customer Ajay has mobile number as: 9807765488
>>>
```